Christian-Albrechts-Universität zu Kiel
Intelligent Systems Lab



Data Drift Detection in Traversability Estimation for Safe Mobile Robot Navigation

Master's Thesis

Luca Mertens

First Supervisor: Prof. Dr.-Ing. Sven Tomforde
Second Supervisor: Nils Bischoff, M.Sc.

Kiel, September 2026

Eigenständigkeitserklärung (Declaration of Authorship)

Hiermit versichere ich, dass ich die Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass ich die Arbeit in keinem anderen Prüfungsverfahren eingereicht habe. Weiterhin erkläre ich, dass die digitale Fassung dieser Arbeit, die dem Prüfungsamt per E-Mail zugegangen ist, der vorliegenden schriftlichen Fassung entspricht.

Kiel, den 10.09.2026 Luca Martens

Abstract

Accurate traversability estimation is essential for safe autonomous navigation. Learned models often show overconfidence in environment types they were not trained for, which in the worst case leads to misclassification of hazardous terrain. This work combines a method from the field of traversability estimation with multiple methods from the field of out-of-distribution (OOD) detection. More concretely, we investigate how the

For this, we train the WayFASTER architecture by M. V. Gasparino *et al.* on the forest recordings of the GrandTour dataset under self-supervised labels. We find that the estimator does not generalize across recordings, which our analysis attributes to the severely imbalanced label distribution. Since established mitigation attempts for imbalanced data do not address this issue, we train the estimator on a small set of relative human labels, which is surprisingly effective.

On the resulting estimator we compare six post-hoc detectors along two axes of novelty: visual and semantic. No single detector performs well on both axes, and the failure modes are complementary. An equally weighted ensemble of three detectors is the only method that separates every group of out-of-distribution recordings from in-distribution ones.

Contents

Eigenständigkeitserklärung (Declaration of Authorship) **1**
Abstract **2**
1 Introduction **6**
1.1 Motivation 7
1.2 Research Objective 7
1.3 Research Questions 8
1.4 Thesis Structure 8
2 Background **9**
2.1 Traversability Estimation 10
2.2 Self-Supervised Label Generation 10
2.3 Uncertainty and Out-of-Distribution Detection 10
2.3.1 Covariate and Semantic Shift 10
2.4 Area Under the Receiver Operating Characteristic Curve (AUROC) 11
2.4.1 The Sensitivity-Robustness Dilemma 11
3 Related Work **12**
3.1 Self-Supervised Traversability Estimation 13
3.1.1 Wild Visual Navigation (WVN) 13
3.1.2 How Does It Feel? 13
3.1.3 Follow the Footprints 13
3.1.4 V-STRONG: Visual Self-Supervised Traversability Learning for Off-road Navigation 13
3.1.5 STEPP: Semantic Traversability Estimation using Pose Projected Features 14
3.2 OOD Detection and Uncertainty Quantification 14
3.2.1 Post-hoc Heuristics (Inference Time) 14
3.2.2 Latent Space and Density Estimation 14
3.2.3 Bayesian Methods 14
3.2.4 Evidential Deep Learning (EDL) 15
3.2.5 Large Pre-trained Models in OOD Detection 15
3.3 Uncertainty-Aware Traversability Estimation 16
3.4 Outdoor Datasets on Quadrupeds 16
3.4.1 TartanGround 16
3.4.2 GOOSE 16
3.4.3 ForEnt 16
4 The GrandTour Dataset **17**
4.1 Preprocessing and Encountered Issues 18
4.1.1 Camera Choice 18
4.1.2 Depth Images: FoundationStereo 18
4.2 Selecting a Train / Validation Split 19
4.2.1 Measurement of Overlap 19
4.2.2 Selected Train/Validation Split 19
5 The WayFASTER Traversability Estimator **20**
5.1 The WayFASTER Architecture 21
5.2 Rationale for Choosing WayFASTER 22
6 Proprioceptive Self-Supervised Labels on GrandTour **23**
6.1 WayFASTER's Labeling Concept 24
6.1.1 Kinematic Model 24
6.1.2 Receding Horizon Optimization 24

6.2 Implementation Details 25
6.2.1 Label Observability 25
6.3 Label Trust 26
7 Training the WayFASTER Estimator on the GrandTour Dataset 27
7.1 Implementation 28
7.1.1 Verification and Bug Fixes 28
7.1.2 Software and Hardware 28
7.2 Estimator Evaluation 29
7.2.1 Evaluation Criteria 29
7.2.2 Reported Metrics 29
7.2.3 Checkpoint Selection and Training Monitor 30
7.3 Training Results 30
8 Failure Analysis of the Trained Estimator 32
8.1 Hypothesis for the Failure 33
8.2 Mitigating Overfitting and Memorization 33
8.2.1 Noise Floor of the Evaluation 33
8.2.2 Regularization Methods 33
8.2.3 Methods Not Included 35
8.2.4 Results of Regularization Experiments 35
8.3 Reproducing WayFASTER on Its Own Dataset 35
8.4 Matching WayFASTER's Labels to the GrandTour Distribution 37
8.5 Overlap of Train/Validation/Test Environments of WayFASTER 38
8.6 Conclusion and Answer to RQ1 40
9 Mitigation: Using a Relative Human Labeling Scheme 41
9.1 Rationale for a Relative Labeling Scheme 42
9.2 W-RIZZ: A Weakly-Supervised Framework for Relative Traversability Estimation in Mobile Robotics 43
9.2.1 W-RIZZ's Labeling Procedure 43
9.2.2 W-RIZZ's Loss Function 44
9.2.3 Evaluation of Estimators Trained on Relative Labels 44
9.3 The Human Pairwise Label Set 45
9.3.1 Annotation Procedure 45
9.3.2 Scale of the Human Label Set 45
9.3.3 Measurement of Self-Consistency 46
9.3.4 Disagreement with RHE Labels 46
9.4 Training Results 47
9.4.1 Training Procedure 47
9.4.2 Results 47
10 Application of OOD Detection Methods 48
10.1 OOD Tiers 49
10.2 OOD Evaluation Metrics 51
10.2.1 AUROC 51
10.2.2 False Positive Rate at 95% True Positive Rate (FPR95) 51
10.3 Methods Under Comparison 52
10.3.1 RQ2: Mahalanobis++ Distance on Network Features 52
10.3.2 RQ3: Mahalanobis++ Distance on DINO Features 54
10.3.3 RQ4: Depth Head Predictive Entropy 55
10.3.4 RQ5: Disagreement Under Viewpoint Change 56
10.3.5 RQ6: Modality Disagreement Between RGB and RGB+Depth 56

10.4 Results of the OOD Detection Methods 57
10.4.1 Performance on the Held-Out ID Set 57
10.4.2 OOD Detection Results 58
10.5 Failure Mode Analysis and Discussion 60
10.5.1 Logit-Mahalanobis++ 61
10.5.2 Methods Based on Visual Features (Encoder-Mahalanobis++, DINOv3 Mahalanobis++) 61
10.5.3 Depth Entropy 62
10.5.4 Viewpoint Disagreement 62
10.5.5 Modality Disagreement 63
10.6 Ensembling to Mitigate Failure Modes 64
10.6.1 Ensemble-Method 64
10.6.2 Results of the Ensemble 65
10.7 Conclusion from the Failure Mode Analysis 65
10.8 Answers to RQ2-RQ6 66
11 Conclusion and Future Work 67
11.1 Conclusion 68
11.2 Takeaways 68
11.3 Limitations of the Work 68
11.4 Future Work 69
11.4.1 Establish a Traversability Estimation Benchmark 69
11.4.2 Use Novel Datasets with More Low-Traversability Examples 69
11.4.3 Compare Different OOD Methods for Traversability Estimators That Rely on the *Unknown* → *Untraversable* Assumption 69
11.4.4 Augment Proprioceptive Labels With Relative Human Labels 69
Bibliography 70
Declaration on the Use of Generative AI 74
1 RHE Hyperparameter Tuning 75
2 GrandTour Tint Correction 76

1 Introduction

1.1 Motivation

Reliable traversability estimation is essential for ground robots in unstructured, off-road environments, where robot-terrain interactions are complex and difficult to label at scale. An example of a traversability estimator is shown in Figure 1. Approaches that derive traversability from the geometry of the ground alone fail in two directions in such environments. In vegetation, they are over-conservative: A patch of tall grass has the same geometry as a solid obstacle, so the robot avoids terrain it could cross. On deformable and slippery ground, they are over-confident: Snow, mud, and wet rock are geometrically flat and are therefore reported as traversable, although the robot can sink in or slip. Both failure modes show that traversability also depends on semantic properties of the terrain, which motivates the use of neural methods that can learn to predict traversability from appearance.

Learned estimators predict traversability from appearance, and recent self-supervised methods obtain their labels from the robot's own experience instead of from human annotation [1], [2]. At the same time, novel datasets for mobile robotics emerge, spanning both synthetic [3] and real-world captures [4].

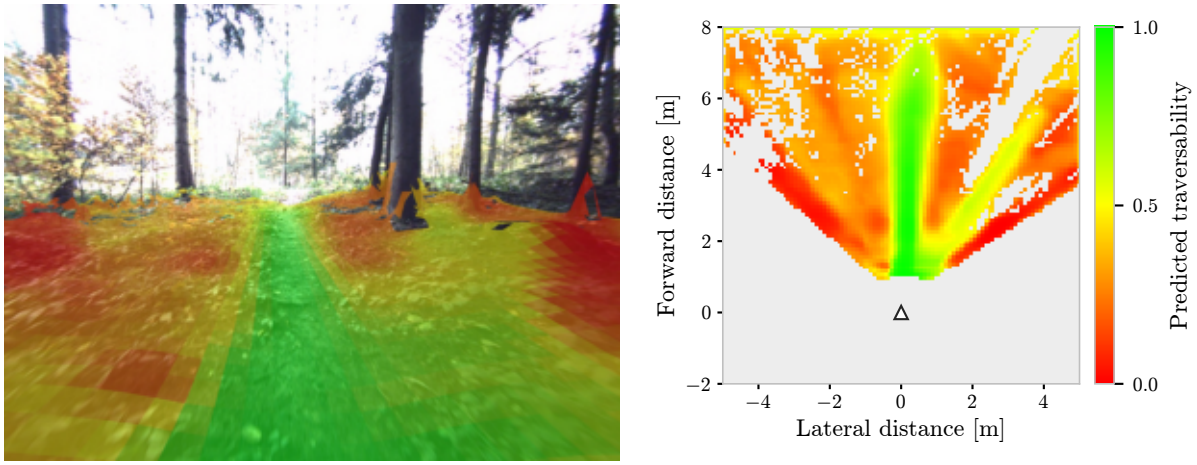


Figure 1: An example of the output of a traversability estimator. Left: the traversability predictions. Right: the same data, shown as a bird's-eye-view map.

A trained estimator covers only the terrain that its training recordings represent. In deployment, it may be exposed to environments these recordings do not cover. For example, an estimator trained on forest recordings may encounter a construction site, snow-covered ground, or an indoor scene. Inputs from such environments are termed *out-of-distribution* (OOD).

The estimator nevertheless returns a traversability value for them, and that value does not indicate whether training data supports it: Deep models are commonly trained and evaluated under a closed-world assumption, and they remain confident on inputs outside their training distribution. A downstream planner can therefore not distinguish an unsupported prediction from a well-supported one, which makes undetected OOD inputs a safety concern. OOD detection provides the missing signal: It indicates whether an input is still covered by the training distribution. The task is intrinsically hard, since the set of possible unknowns cannot be modeled exhaustively.

1.2 Research Objective

This thesis asks how a traversability estimation network can be augmented to detect novel, out-of-distribution terrain. The goal is to detect drift in the input distribution of an already trained estimator: The estimator predicts traversability, and a detector placed on top of it reports whether that prediction rests on familiar ground.

Reaching that point requires an estimator to attach the detectors to, so the work has two parts: training a traversability estimator on the GrandTour dataset [4] (RQ1), and comparing OOD detection methods on the resulting estimator (RQ2-RQ6). We initially planned to obtain that estimator from self-supervised proprioceptive labels, which are derived from the robot's own experience and therefore require no human annotation.

Since the first part did not succeed under that plan, we changed the supervision and trained an estimator on relative human labels (Chapter 9).

The objective itself is unaffected by the change, since we restrict the comparison to post-hoc detectors, which read a trained estimator regardless of how it was supervised.

1.3 Research Questions

RQ1 Can the WayFASTER architecture, supervised by proprioceptive labels, be trained on the GrandTour dataset such that it generalizes across recordings?

RQ2-RQ6 ask the same of one method family each: Does its score separate in-distribution from shifted terrain on the trained estimator, and how does it fail?

RQ2 A distance-based method on the estimator's own features (Section 10.3.1).
RQ3 A distance-based method on frozen DINOv3 features (Section 10.3.2).
RQ4 The predictive entropy of the estimator's depth head (Section 10.3.3).
RQ5 The disagreement between predictions of the same location under viewpoint change (Section 10.3.4).
RQ6 The disagreement between the RGB-only and the RGB+depth inference path (Section 10.3.5).

1.4 Thesis Structure

Chapter 2 introduces basic concepts used throughout the thesis, and Chapter 3 briefly surveys the fields this work builds on. Chapter 4 presents the dataset and Chapter 5 the architecture used for traversability estimation. Chapter 6 presents the proprioceptive labeling scheme, and Chapter 7 shows the training results of the estimator under that supervision. Chapter 8 diagnoses why the estimator does not generalize and answers RQ1. Chapter 9 presents an alternative labeling scheme and trains a second estimator from these labels. Chapter 10 compares the OOD detection methods on that estimator and answers RQ2-RQ6. Chapter 11 summarizes the findings, states the limitations, and names directions for future work.

2 Background

2.1 Traversability Estimation

Traversability describes how well a robot can move over a given piece of ground. A traversability estimator predicts this property from the robot's sensors, ahead of the robot and before it makes contact. The estimator used in this thesis (Chapter 5) consumes a *window* of consecutive frames, each holding an RGB image, a depth image, and the robot pose from odometry. It predicts a bird's-eye view (BEV) grid centered on the robot, i.e., a view of the ground from above, discretized into square *cells* of fixed size. Each cell carries a linear traversability value $\mu \in [0, 1]$ and an angular one $\nu \in [0, 1]$.

A local planner consumes the grid as a cost map: It scores candidate trajectories by the cells they cross and prefers those with high traversability.

2.2 Self-Supervised Label Generation

Training such an estimator requires traversability labels of some sort. Hand-annotated labels are expensive to collect and are often inconsistent (see Chapter 9 for more details), which is why self-supervised methods are popular in the literature.

The following two families are especially common (Chapter 3). *Proprioceptive pseudo-labels* measure the interaction while driving, for example the bandpower of the vertical IMU acceleration or the slip between commanded and measured motion, and assign the measured value to the cells the robot occupied. *Trajectory-derived positives* treat the traversed footprint as traversable and everything unvisited (or anomalous) as untraversable.

Our chosen method, WayFASTER, uses the first family and is described in Chapter 7.

2.3 Uncertainty and Out-of-Distribution Detection

For out-of-distribution (OOD) detection, it is important to distinguish between different types of uncertainty:

Aleatoric Uncertainty (Data Uncertainty) Aleatoric is defined as "left to or resulting from a chance process" [5]. Aleatoric uncertainty refers to the intrinsic randomness present in the data generation process. It captures variability that cannot be reduced simply by collecting more data. For example, when a mobile robot repeatedly traverses the same gravel patch under identical conditions, the measured wheel slippage will vary across runs, producing a distribution of values rather than a single consistent outcome. An uncertainty-estimator *should* predict high aleatoric uncertainty for rough terrain in this case. This is not a failure of the estimator. It is a property of the environment.

Epistemic Uncertainty (Model Uncertainty) Epistemic can be defined as relating to "knowledge, [...], or degree of validation" [6]. This captures the lack of knowledge in the model parameters θ due to the sparsity of training data. High epistemic uncertainty is the primary signal for OOD detection. It indicates that the predicted cost (and even the predicted aleatoric uncertainty) is untrustworthy.

2.3.1 Covariate and Semantic Shift

In realistic deployments, a model may encounter *domain shift*, where the test environment differs from the training domain. The OOD literature distinguishes two forms of shift [7], [8]: Covariate shift and semantic shift. Under *covariate shift*, the input distribution changes ($P_{\text{train}(X)} \neq P_{\text{test}(X)}$) while the true conditional distribution remains unchanged ($P_{\text{train}(Y \mid X)} = P_{\text{test}(Y \mid X)}$). For example, a paved street looks very different from the forest terrain we train on, but is still trivially traversable. Under *semantic shift*, the input-output relation itself changes ($P_{\text{train}(Y \mid X)} \neq P_{\text{test}(Y \mid X)}$). Strictly, semantic shift is defined over a closed label set, whereas traversability is a continuous property of the ground, so we use the term for a changed input-output relation. For example, forest terrain with higher grass than in the training set looks familiar, but is harder to traverse than the model expects. Both forms usually

occur together: A model trained in forest terrain and deployed on a construction site encounters a novel appearance as well as a novel input-output relation. Either case yields out-of-distribution (OOD) inputs that lie beyond the model's training experience. Section 10.1 turns the two forms into the axes along which we group the evaluation recordings.

2.4 Area Under the Receiver Operating Characteristic Curve (AUROC)

AUROC measures the **A**rea **U**nder the **R**eceiver **O**perating **C**haracteristic curve, and is the most commonly reported metric in OOD detection research [7], [9], [10]. The ROC curve plots the true positive rate (TPR) against the false positive rate (FPR) at different decision thresholds (see Figure 2), and the AUROC then aggregates the ROC over every possible threshold.

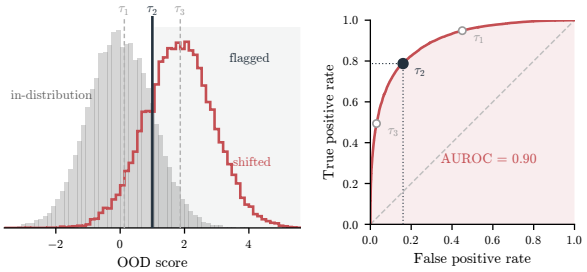


Figure 2: Detection read as a binary decision on a score. [11] Left, the in-distribution and the shifted score distributions overlap, with a threshold τ drawn in: Everything to its right is flagged as OOD. Right, each of the three candidate thresholds is one point on the ROC curve, and the shaded area under that curve is the AUROC. The dashed diagonal marks chance-level separation.

Formally, for a binary classifier with uncertainty scores $s(x)$ and ground truth labels $y \in \{0, 1\}$ (where 1 denotes OOD), the AUROC is defined as:

$$\text{AUROC} = \int_0^1 \text{TPR}(\tau)\, d(\text{FPR}(\tau))$$

where τ represents the decision threshold. An AUROC of 1.0 indicates perfect separation between ID and OOD samples, while 0.5 corresponds to random guessing.

AUROC's main advantage is that it is threshold-free: It aggregates over every operating point, so two detectors can be compared without first committing to a decision threshold. Equivalently, it is the probability that a randomly drawn OOD sample receives a higher score than a randomly drawn ID sample [11]. However, that same property means AUROC says nothing about which threshold a deployed detector should use.

2.4.1 The Sensitivity-Robustness Dilemma

A central trade-off in the field is that between OOD generalization ability (predictions should remain confident and robust under distributional shifts like sensor noise or sim-to-real) and OOD sensitivity to distributional shifts that should be detected (e.g., due to novel terrain) [12]. For an off-road robot, methods that are too sensitive might flag every shadow or lighting change as OOD, effectively paralyzing the robot. Methods that are too 'robust' may generalize over dangerous obstacles.

3 Related Work

This chapter covers the three areas this thesis combines: self-supervised traversability estimation, OOD detection and uncertainty quantification, and methods that address both at once. Finally, we discuss datasets that were considered as alternatives to our chosen dataset (GrandTour).

3.1 Self-Supervised Traversability Estimation

The methods described here share the principle introduced in the background chapter: The labels are derived from the robot's own experience instead of human annotation.

3.1.1 Wild Visual Navigation (WVN)

J. Frey *et al.* [13] propose WVN, a vision-only system that learns traversability online, during deployment. The architecture separates a frozen offline part from a small online part, so that only the latter has to be trained on the robot.

The offline part is a DINO-ViT [14], pre-trained in a self-supervised manner and then kept frozen in the field. It extracts dense features from monocular RGB images. Since storing and processing these dense features on a mobile robot is too expensive, each image is divided into 100 SLIC superpixels [achantaSLICSuperpixelsCompared2012], and the features are averaged per superpixel, which yields one 90-dimensional embedding per segment. The online part that is trained during deployment is a two-layer MLP on top of these embeddings. Its supervision comes from a short human demonstration: An operator teleoperates the robot over terrain it can handle, and each traversed segment receives a traversability score derived from the velocity tracking error of the locomotion controller, i.e., the deviation between the commanded and the estimated velocity.

The method bypasses the issue of sparse negative labels (i.e., low-traversability terrain): Segments that are both unvisited and anomalous are labeled as untraversable. For this, the reconstruction loss of an autoencoder is used as an anomaly signal.

3.1.2 How Does It Feel?

The method by M. G. Castro *et al.* [2] uses a self-supervised learning pipeline that fuses exteroceptive visual data with proprioceptive feedback. The input consists of 2×2 meter bird's-eye view patches containing RGB and height statistics, processed by a ResNet18 backbone. The model also incorporates robot velocity, which is expanded via Fourier Feature Mapping and processed by an MLP to capture speed-dependent dynamics. These streams merge to predict a single scalar traversability cost per pixel. Training relies on "pseudo-ground truth" generated directly from the robot's onboard IMU, specifically the bandpower of vertical acceleration. This allows the model to map visual textures (like grass or gravel) to physical sensations of roughness without manual annotation. The model was only benchmarked against standard geometric occupancy-based navigation stacks and appearance-based roughness estimators. In large-scale field experiments, the proposed method outperformed the geometric baseline, reducing human interventions by up to 57%. In cases where the baseline treated all non-obstacle terrain as equally traversable, the learned model exhibited behaviors such as deviating from straight paths to avoid rough gravel or vegetation.

3.1.3 Follow the Footprints

Y. Jeon *et al.* [15] employ a detection network that fuses visual (e.g., camera input) and geometric (e.g., lidar) data using a Guide Filter Network to preserve structural details in off-road environments. Training relies on a Footprint Supervision Module, which generates self-supervised labels by treating the robot's historical trajectory (its *footprint*) as positive ground truth for traversable terrain.

3.1.4 V-STRONG: Visual Self-Supervised Traversability Learning for Off-road Navigation

The Segment Anything Model (SAM) [16] is a vision transformer for video segmentation, which can be controlled ("prompted") by supplying positive and negative points of the object to be segmented. S. Jung *et al.* [17] use SAM for self-supervised traversability estimation: Instead of treating only the exact vehicle footprint as traversable, the method uses trajectory points as "prompts" to query SAM masks, effectively propagating positive labels to coherent semantic regions (e.g., vehicle footprint $\rightarrow$ the

entire width of a dirt road). The authors claim that the method reaches state-of-the-art performance, especially w.r.t. generalization.

3.1.5 STEPP: Semantic Traversability Estimation using Pose Projected Features

S. Ægidius *et al.* [18] propose STEPP, which frames traversability as an anomaly detection problem rather than as a traditional segmentation or classification problem.
The system first extracts dense, pixel-wise feature embeddings from the input image using a pre-trained DINOv2 Vision Transformer [19]. To link these semantic features to specific terrain regions, the image is divided into superpixels via SLIC [20]. Embeddings from superpixels that overlap with the robot's projected path are then averaged into compact feature vectors, which are used to train an Encoder-Decoder MLP to reconstruct features of known "safe" terrain.
At inference time, the model tries to reconstruct the features of every observed segment. Segments it reconstructs well (indicated by a low reconstruction error) are considered traversable since the model has seen similar terrain before. Segments with high reconstruction errors are flagged as obstacles or anomalies.

3.2 OOD Detection and Uncertainty Quantification

The research area of out-of-distribution (OOD) detection is extensive. Consequently, there are multiple survey articles, e.g., [10], [21], [22], each proposing its own scheme for categorizing OOD detection approaches. We classify them as follows:

While OOD detection determines whether an input originates from outside the training data distribution, it closely intersects with Anomaly Detection (AD) [23] and Anomaly Localization (AL) [24]. AD operates at the instance level to identify deviations from a single nominal distribution (e.g., detecting a defective product), whereas AL extends this to fine-grained spatial or temporal pinpointing of the anomalous regions within that instance (e.g., segmenting pixel-level defects). Although AD and AL are related fields, this thesis focuses on OOD detection.

3.2.1 Post-hoc Heuristics (Inference Time)

These methods require no retraining of the model. They derive an OOD score from a fixed, pre-trained network. For example, simple baselines like Softmax Entropy or Maximum Softmax Probability (MSP) estimate uncertainty directly from classifier outputs [25]. While computationally negligible, these methods can suffer from overconfidence on OOD samples, and they would need to be slightly adapted to regression tasks.

More advanced post-hoc methods analyze the feature representations learned by the network rather than just the final output. The Mahalanobis distance can detect anomalies by measuring the distance of test samples to class-conditional Gaussian distributions in the latent space [26]. Recent variants, such as the Relative Mahalanobis Distance, further improve sensitivity to near-OOD samples by correcting for background statistics [27].

3.2.2 Latent Space and Density Estimation

As an alternative to analyzing the features of a discriminative classifier, generative models can be employed to explicitly model the distribution of the training data. Models like Variational Autoencoders (VAEs) or Normalizing Flows estimate the training density distribution directly, flagging low-density samples as anomalies [28]. These approaches rely on the assumption that OOD data will lie in low-probability regions of the learned manifold.

3.2.3 Bayesian Methods

Bayesian approaches aim to learn a distribution over model parameters instead of fixed weights (e.g., for a simple MLP, each scalar weight could conceptually be replaced by a Gaussian distribution). Since

learning model weight distributions is intractable, these methods instead try to approximate Bayesian inference. One popular approach is Monte Carlo Dropout, where a distribution is built at inference time by evaluating the model k times, each time zeroing weights at random [29]. This enables capturing epistemic uncertainty without requiring multiple training runs. However, MC Dropout often requires careful tuning to yield calibrated uncertainty estimates [30]. Another approach is ensembling, which involves training multiple independent models with different initializations and measuring predictive disagreement (e.g., Deep Ensembles) [31]. While effective at capturing epistemic uncertainty, ensembles significantly increase computational cost, both in training and inference. Even existing attempts to make them computationally more efficient, such as distilling ensembles into a single model [32], are often infeasible for continuous learning use cases or resource-constrained environments.

3.2.4 Evidential Deep Learning (EDL)

Standard neural classifiers typically output a single probability vector using a softmax function. A major limitation of this approach is that the model often yields high-confidence predictions even for ambiguous or out-of-distribution inputs. To mitigate this without incurring the high computational cost of sampling-based Bayesian methods, Evidential Deep Learning (EDL) [33] introduces a deterministic approach to uncertainty estimation.

Instead of predicting class probabilities directly, EDL predicts a non-negative *evidence* vector $\boldsymbol{e} \in \mathbb{R}^K_{\geq 0}$ for a K-class problem in a single forward pass. This evidence is generated via a ReLU or softplus activation and parametrizes a Dirichlet distribution over the categorical probability vector $\boldsymbol{p}$. The parameters $\boldsymbol{\alpha}$, the expected class probabilities $\hat{\boldsymbol{p}}$, and the associated epistemic uncertainty u are defined as:

$$\boldsymbol{\alpha} = \boldsymbol{e} + \mathbf{1}, \quad \hat{p}_k = \frac{\alpha_k}{S}, \quad u = \frac{K}{S}, \quad \text{where} \quad S = \sum_{i=1}^{K} \alpha_i.$$

Here, the total strength S is inversely related to the uncertainty: High evidence results in a larger S and, consequently, lower uncertainty.

Deep Evidential Regression [34] adapts this framework for continuous targets. Rather than predicting a point estimate for the target y, the network outputs the hyperparameters of a higher-order evidential distribution that governs the target likelihood. This enables efficient, single-pass uncertainty estimation for regression tasks, avoiding the computational overhead associated with ensemble methods.

3.2.5 Large Pre-trained Models in OOD Detection

Recent OOD detection approaches increasingly rely on large pre-trained foundation models, which use extensive unlabeled or multimodal data to detect anomalies without task-specific retraining [9], [35].

Vision Transformers Self-supervised vision transformers, such as DINO [14], provide embeddings that are said to be less sensitive to local noise than traditional CNNs. For traversability estimation, a frozen DINO backbone typically extracts features. OOD detection is then performed by computing the distance (e.g., Mahalanobis distance) between test features and the training embedding distribution [36]. However, its high inference latency remains a significant challenge for resource-constrained robotics.

Vision-Language Models Vision-Language Models (VLMs) like CLIP [37] learn a joint image-text embedding space via contrastive learning on massive datasets. This allows for classifying terrain using natural language descriptions (e.g., "safe grass") rather than fixed numerical labels. For OOD detection, methods often employ a "dual-pattern" strategy [38]: an input is flagged as an anomaly if it deviates significantly from both the stored visual history of known classes and their textual

embeddings. For instance, encountering black ice after only seeing dry asphalt triggers both a visual feature mismatch and a low textual similarity to prompts like "safe road." Despite their zero-shot capabilities, VLMs require comparably large computational resources for inference and can be sensitive to prompt phrasing [38].

3.3 Uncertainty-Aware Traversability Estimation

EVORA [39] consists of a pipeline that enhances traction prediction by explicitly quantifying uncertainty. Rather than regressing a scalar cost, the model employs Evidential Deep Learning (EDL) (see Section 3.2.4) to predict the concentration parameters of a Dirichlet distribution over discretized traction bins. Intuitively, this output represents a "distribution over distributions." A sharp Dirichlet distribution suggests high confidence in specific traction values, whereas a flat distribution implies conflicting evidence. This formulation allows EVORA to disentangle two distinct types of risk: **Aleatoric uncertainty** is captured by the variance of the predicted Dirichlet distribution, representing inherent data noise or ambiguity in the terrain appearance. The **epistemic uncertainty** is estimated by training a density estimator (a Normalizing Flow) on the feature embeddings. Intuitively, the Normalizing Flow acts as a sequence of invertible transformations that "warps" a simple base distribution (like a Gaussian) until it fits the complex shape of the training data. If a new terrain patch falls outside this learned shape (low likelihood), it is flagged as Out-of-Distribution (OOD).

3.4 Outdoor Datasets on Quadrupeds

We discuss the datasets that come closest to GrandTour [4] in platform and setting since these are the candidates we considered as alternative training data for our label pipeline (Chapter 6).

3.4.1 TartanGround

TartanGround [3] contains 1.5 million samples from 910 simulated trajectories across 70 photorealistic environments, with 360° RGB stereo, depth, lidar, semantics, and occupancy maps. Its quadruped subset is generated by tracking the planned paths with an ANYmal-D [40] in Gazebo, which also yields base velocities and accelerations, joint states, and per-leg contact forces. However, the simulated terrain is rigid geometry, so these proprioceptive readings do not capture the slip of real grass, mud, or undergrowth, which makes it unsuitable for the generation of proprioceptive labels.

3.4.2 GOOSE

GOOSE [mortimerGOOSEDatasetPerception2024] [hagmannsExcavatingWildGOOSEEx2025] contains roughly 21 hours of recordings of unstructured off-road terrain: about 14 hours from an off-road vehicle, 4 hours from a robotic excavator, and 3 hours from a quadruped. The recordings follow terrain the platforms can actually drive on, and therefore carry the same selection bias towards traversable terrain as GrandTour (Chapter 9).

3.4.3 ForEnt

ForEnt [41] is the closest match to the setting of this work: 1.7 km of traversals in 11 sequences with a Unitree Go2 quadruped across eight forest sites, recorded as time-synchronized RGB-D images, lidar scans, and proprioceptive data, with 69 annotated entrapment events. It is not yet released at the time of writing. The authors kindly shared their data with us, but parts required by our pipeline (e.g., certain calibration matrices) were missing, so we could not use it.

4 The GrandTour Dataset

The GrandTour dataset by J. Frey *et al.* [4] is a large-scale dataset for legged robotics. It contains over 9 hours of data collected from a quadrupedal robot, *ANYmal-D* [40], equipped with the *Boxi* payload [42], which carries a variety of sensors, including stereo cameras, depth cameras, lidar, and GNSS. The dataset covers diverse environments, including urban areas, forests, indoor settings, and industrial sites, with a focus on daytime recordings. The most represented environment is a forest slice of 1 h 40 min, which we use as the In-Distribution (ID) set for this thesis. We discuss the dataset's structure, the sensor setup, and the data splits used for training and evaluation in this chapter.

4.1 Preprocessing and Encountered Issues

In the following, we present the most relevant preprocessing steps we applied to the GrandTour dataset.

Missing odometry Both the label estimator and the BEV grid require precise odometry information. The dataset advertises a post-processed, tightly coupled GNSS-inertial solution, but it is unavailable on nine missions, with no apparent pattern. We therefore run the whole pipeline on DLIO lidar odometry [43], which is available everywhere, so that all missions are treated identically. When benchmarked against the total-station ground truth on the 13 missions where it is available, it has a median absolute trajectory error of 14 cm.

Red tint In dark scenes, the auto white balance of the front-center Alphasense camera occasionally fails, producing a strong red tint. It seems to be situation-dependent and cannot be undone by a fixed color transform. The defect is correlated with the mission, making it a spurious cue that the network could exploit. Because most frames in affected missions remain correctly exposed, we correct the tint by matching the luminance and median chroma of flawed frames to nearby white-balanced frames within the same mission (Appendix B).

Excluded mission We exclude the ICE-1 mission, in which the robot was pushed across ice while standing still. The extended sliding violates the kinodynamic model with which our labels are estimated (Chapter 6).

False positives of the anonymizer The dataset uses a face-blurring mechanism, which occasionally produces false positives, resulting in large areas of the terrain being pixelated. We do not exclude these frames, as there is no reliable way to detect them. This is a source of noise in the training data.

4.1.1 Camera Choice

We require RGB images with depth that is synchronized in both time and space, for which GrandTour offers two candidates: the Zed2i RGB-D camera, timestamped over a USB serial buffer and thus, per the authors, "unreliable" [42]; and the Alphasense RGB camera, synchronized via PTP to sub-millisecond accuracy [42]. The robot's RealSense D435 only provides depth here, not RGB images, so it is not a candidate. We use the Alphasense camera for its accurate synchronization, at the cost of depth not being provided directly and having to be generated separately (Section 4.1.2).

4.1.2 Depth Images: FoundationStereo

The lidar point cloud cannot be used as a depth source for the WayFASTER architecture, since its returns are too sparse to fill a per-pixel depth image. We therefore generate depth from stereo images with FoundationStereo [44], a zero-shot stereo-matching model that needs no fine-tuning on our domain. We use the color camera `front_center` as the stereo reference and pair it with the monochrome `front_right`, so that the estimated depth is natively aligned with the RGB image. Using the dedicated monochrome pair would double the distance between the cameras (the baseline), but its depth would then have to be reprojected into the color frame, which leaves disocclusion holes along object edges that cannot be recovered in post-processing.

4.2 Selecting a Train / Validation Split

Due to the limited size of the forest slice of the GrandTour dataset (1 h 40 min, compared to WayFASTER's 9 h 11 min), we split the dataset into training and validation sets only, without a separate test set. The held-out validation set is used both to select the best model checkpoint during training and to evaluate the final model afterwards. This is common practice in the field of traversability estimation, where the final estimator is typically evaluated in closed loop with the physical robot and a planner, as opposed to using a classic test set. We consider this an acceptable trade-off for this work since our primary goal is to demonstrate that the model can learn to predict traversability on the GrandTour dataset, thereby enabling the later OOD-detection stage, rather than to benchmark it against other traversability estimators.

4.2.1 Measurement of Overlap

To make sure that the validation set does not contain any data that the model has already seen during training, we measure the overlap between the training and validation sets in terms of ground coverage. We do this by comparing the ground cells (in world space) observed by the camera in training and validation. If the same cell is observed from multiple viewpoints, it is counted only once. A cell is marked as observed if it falls inside the camera frustum within 8 m. We report two things: the overlap for every angle and the overlap restricted to poses within 120° of the same heading, meaning that a cell seen from the opposite direction is not counted as overlap. To place missions in a common frame, we use the DLIO lidar odometry, as mentioned in Section 4.1, and anchor it in a shared global frame with GNSS readings by performing a rigid 2D fit to GNSS data points with low covariance ($\text{cov} < 0.05\text{m}^2$).

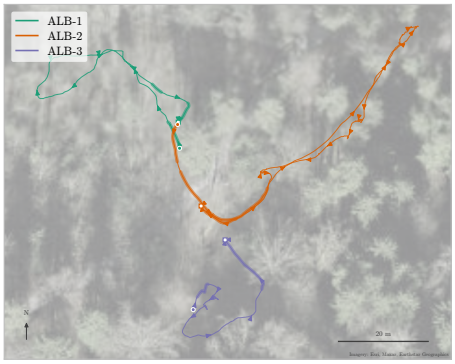


Figure 3: A visualization of the ground overlap between the ALB-* missions. Even though they share the same prefix, most missions do not overlap meaningfully.

Some missions could not be measured because they did not have enough accurate GNSS data points. All these unmeasured missions are placed in the same set.

4.2.2 Selected Train/Validation Split

Table 1 shows the statistics of our selected train/validation split.

Split	Hours (% of ID)	Hours	$\mu < 0.3$	$0.3 \leq \mu < 0.6$	$\mu \geq 0.6$	Leakage of train into valid (% of observed area)	
						any angle	$\leq 120°$ heading Δ
Train	73.2%	1 h 13 min	1.2%	2.1%	96.7%	2.1%	0.8%
Valid	26.8%	27 min	1.4%	2.9%	95.6%		

Table 1: Our selected train/validation split, with traversability (μ) distribution statistics. We selected the training and validation split based on the following criteria: (a) The split ratio should be close to 80/20. (b) The μ distributions should be similar between train and validation. (c) Leakage of train data into the validation set should be minimized.

5 The WayFASTER Traversability Estimator

The WayFASTER architecture by M. V. Gasparino *et al.* [1] fuses *semantic* features extracted from RGB images via a ResNet34 backbone with *geometric* occupancy data derived from depth images. It is presented in this chapter.

5.1 The WayFASTER Architecture

Figure 4 shows the architecture, which processes the two input modalities in separate branches. Both branches consume the same window of six frames.

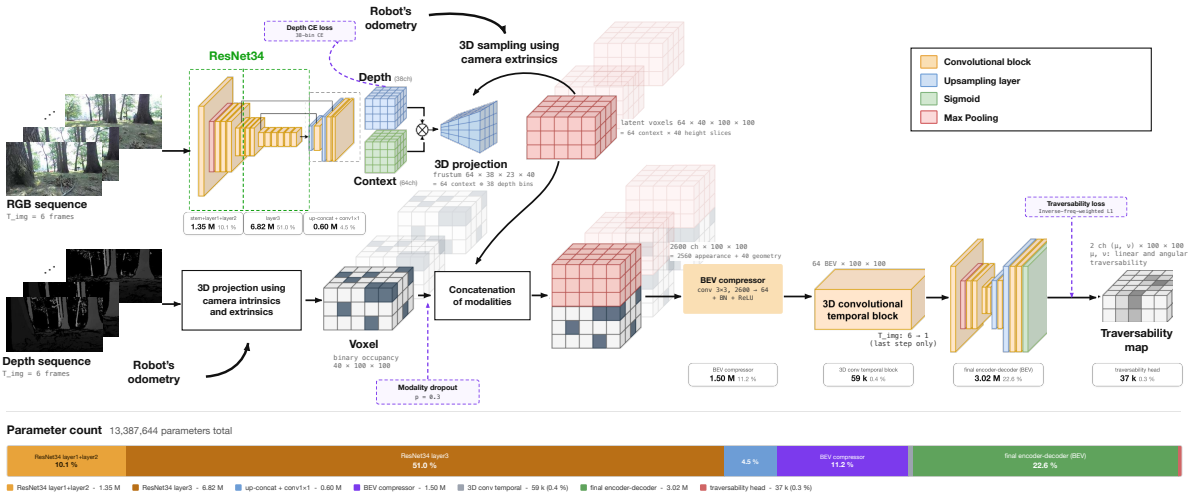


Figure 4: The WayFASTER architecture, as described in M. V. Gasparino *et al.* [1].

The upper branch encodes the RGB images with a ResNet34 backbone into two per-pixel outputs: a discrete depth distribution over 38 bins, and a 64-channel semantic feature vector (the *context* vector in the figure). Taking the outer product of the two distributes every context vector over the depth bins in proportion to their predicted probability, which yields a frustum of features in camera coordinates. The frustum is then resampled into a voxel grid of 40 height slices over a 100×100 bird's-eye view (BEV) grid, i.e., a view of the 3D space from above, using the camera extrinsics and the robot's odometry.

The lower branch projects the measured depth images into the same voxel grid using the camera intrinsics and extrinsics, where they form a binary occupancy grid. It therefore carries geometry, but no appearance information.

Concatenating the two branches yields 2560 appearance channels and 40 geometry channels per BEV cell, which are compressed to 64 channels. The subsequent layers can, in theory, choose between the semantic features of the upper branch and the geometric features of the lower branch when predicting traversability. During training, the depth branch is dropped with a probability of 0.3, so that the network learns to also predict from RGB alone.

A 3D convolutional block then fuses the six timesteps into the most recent one. The six frames span 2.5 s (at 2 Hz), which is the length of the model's short-term memory. An encoder-decoder and a sigmoid head map the fused features to the output: a 100×100 BEV image with one channel for linear (μ) and one for angular (ν) traversability. Most of the 13.4 M parameters sit in the ResNet34 backbone.

5.2 Rationale for Choosing WayFASTER

The primary reason for selecting WayFASTER is its use of physical slippage (the discrepancy between desired and actual control commands) as a self-supervision metric, which directly aligns with what we aim to optimize through traversability estimation. The paper was also peer-reviewed and accepted at ICRA'24, which gave us some confidence in the soundness of its design and reported results.

Architecturally, WayFASTER combines well-understood components (ResNets, 3D convolutions), and its intermediate representations offer several natural places on which OOD detection methods can be applied.

From a practical standpoint, the architecture runs on contemporary on-board hardware common in robotics, such as the NVIDIA Jetson. The training implementation is open-source[1], and the original dataset is openly available along with its labels, which lets us reproduce the published results as a sanity check for our own pipeline later (Section 8.3).

[1]The labeling process is not open-source, though.

6 Proprioceptive Self-Supervised Labels on GrandTour

While the WayFASTER architecture and training code are open source, the self-supervised label generation pipeline is not. We therefore reimplemented the label generation method based on the description in [45] and [1]. This proved to be a non-trivial task since the original description is relatively high-level and omits many details and hyperparameters. We first introduce the method, describe our implementation, and provide a summary of the resulting label set.

6.1 WayFASTER's Labeling Concept

The basic idea of the labeling method is to use a simple kinematic model (i.e., a unicycle model) to predict the robot's trajectory based on its commanded velocities. The predicted trajectory is then compared to the actual trajectory measured by the robot's odometry. If the actual trajectory deviates from the predicted one, it is assumed that the terrain is less traversable, and the label is adjusted accordingly.

6.1.1 Kinematic Model

The kinematic model is described by the following equations. To calculate the next position $p_{x,k+1}$ from the current position $p_{x,k}$, the commanded linear velocity v_k, and the heading angle θ_k, we use the following equations:

$$p_{x,k+1} = p_{x,k} + \mu v_k \cos\theta_k$$
$$p_{y,k+1} = p_{y,k} + \mu v_k \sin\theta_k$$

Note the scale factor μ on the commanded linear velocity. This is an optimization variable that is adjusted to minimize the difference between the predicted and actual trajectory, and it serves as our linear-traversability label. If μ is close to 1, the predicted trajectory is a good approximation of the actual trajectory. If μ is significantly different from 1, it indicates a discrepancy between the model and the real-world behavior.

The same is done for the heading angle θ_{k+1}, which is updated based on the commanded angular velocity ω_k and the angular traversability factor ν:

$$\theta_{k+1} = \theta_k + \nu\omega_k$$

6.1.2 Receding Horizon Optimization

A receding horizon estimator (RHE) fits a model to a sliding window of the most recent measurements. Solving this problem once yields the states and the parameters that best explain that window. The window then advances by one step and the problem is solved again, with the previous solution entering the next one as the prior $\tilde{x}_k, \tilde{m}$. Every timestep therefore gets its own (μ, ν), which is the label. The first two terms of the objective are what tie the windows together: the state and the parameters may move between consecutive solutions only as far as the measurement residual justifies, weighted by P_x and P_m. The horizon length N sets the trade-off since a long window averages out odometry noise, while a short one follows genuine terrain changes more quickly.

The two factors are estimated over a window of the N most recent steps by solving

$$\min_{x_{k:k+N},m} \underbrace{\|x_k - \tilde{x}_k\|^2_{P_x}}_{\substack{\text{State continuity}\\ \text{between windows}}} + \underbrace{\|m - \tilde{m}\|^2_{P_m}}_{\substack{\text{Label should not change}\\ \text{too much at once}}} + \underbrace{\sum_{i=k}^{k+N} \|x_i - h(z_i)\|^2_{P_w}}_{\text{Reconcile model with odometry via } \mu,\nu}$$

subject to the constraints

$$x_{k+1} = f(x_k, u_k), \quad \mu, \nu \in [0, 1],$$

where $x_k = \left[p_{x,k}, p_{y,k}, \theta_k\right]^\top$ is the state, $u_k = [v_k, \omega_k]^\top$ the commanded velocities, f the kinematic model above, $m = [\mu, \nu]^\top$ the parameter vector, $\tilde{x}_k$ and $\tilde{m}$ the state and parameters carried over from the previous window, $h(z_i)$ the measurement model that maps the odometry measurement z_i to a pose, and P_x, P_m, P_w weighting matrices. The parameters are assumed constant within a window.

6.2 Implementation Details

The unicycle model of Section 6.1 has no term for lateral velocity, while the quadruped can also be commanded to move sideways. Such motion appears in the odometry but not in the prediction, so the estimator would have to absorb it into μ and ν, which distorts the label. To quantify the impact of this, we measure how often each channel of the twist command is used (Figure 5): The sideways channel is practically never commanded, while the forward and the yaw channel are active during a large share of every mission. We hence consider the error introduced by the missing term negligible and keep the unicycle model.

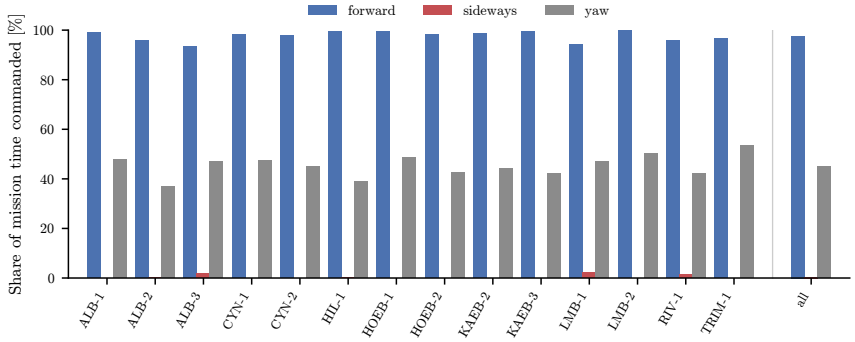


Figure 5: Share of mission time each twist command channel is commanded above 0.01, over the 14 missions of our train/validation split.

6.2.1 Label Observability

A parameter is only identifiable while the command that drives it is active: If the commanded linear velocity is zero, the predicted trajectory does not depend on μ, and the estimator returns the prior $\tilde{m}$ rather than a value supported by the measurements. The same holds for ν and the commanded yaw rate. Since the estimator emits a label for every timestep regardless, we attach an observability score to each label, based on the average magnitude of the command in the current window.

To quantify observability, we use a cubic smoothstep function[2] as a ramp between two thresholds u_{lo} and u_{hi}:

$$w(\overline{u}) = \begin{cases} 0 & \overline{u} \le u_{\text{lo}} \\ 3t^2 - 2t^3 & u_{\text{lo}} < \overline{u} < u_{\text{hi}} \\ 1 & \overline{u} \ge u_{\text{hi}} \end{cases}$$

$$t = \frac{\overline{u} - u_{\text{lo}}}{u_{\text{hi}} - u_{\text{lo}}},$$

where $\overline{u}$ is the mean absolute command magnitude over the window. The function is visualized in Figure 6. We evaluate it separately for both parameters: $o_\mu = w(\overline{v})$ from the commanded linear velocity, and $o_\nu = w(\overline{\omega})$ from the commanded yaw rate.

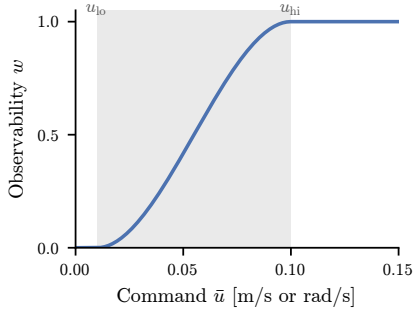


Figure 6: The observability ramp. We use $u_{\text{lo}} = 0.01$ and $u_{\text{hi}} = 0.1$, meaning that the label is only fully trusted when the command velocity is over 0.1 m/s or rad/s.

During training, we multiply the loss for each label by its observability score, which effectively gates out labels that are not supported by the measurements. Figure 7 shows how much loss weight the gate leaves per mission. Most states keep their full weight for the linear traversability label μ, while a large share of them is gated out entirely for the angular traversability label ν since the yaw rate is rarely commanded far above u_{hi}. We therefore consider the angular traversability label unreliable and do not report ν in this work.

[2] As commonly used in computer graphics.

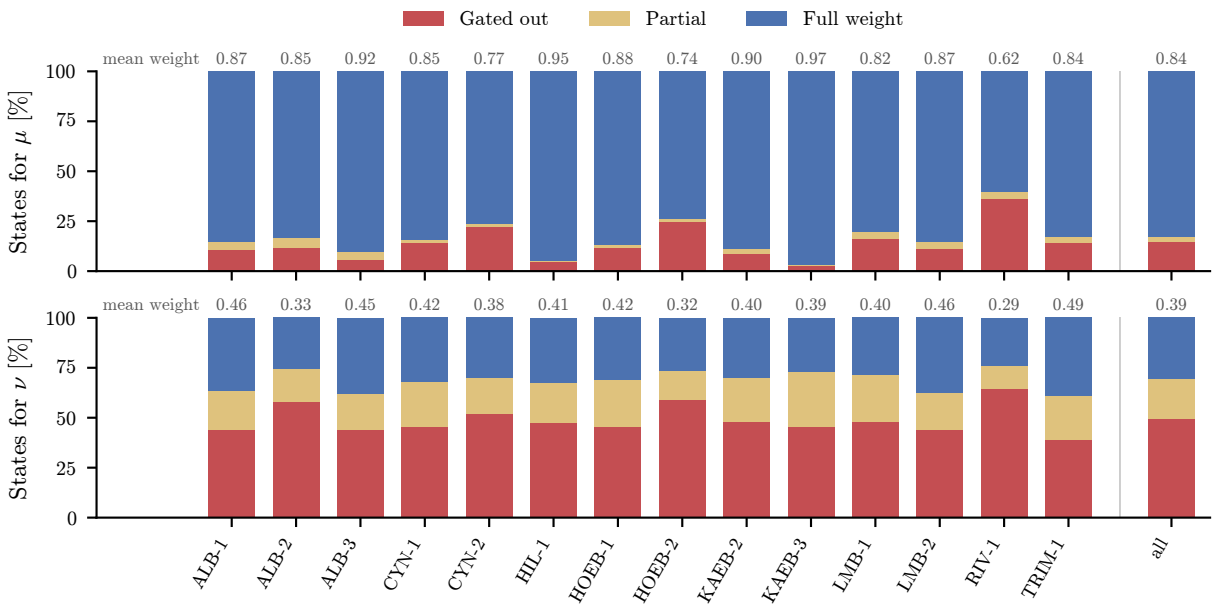


Figure 7: Impact of the observability weighting on the loss per mission.

6.3 Label Trust

Every result in this work rests on the labels: If they are wrong, so is everything trained or evaluated on them. No ground truth exists to check them against, since the labels are themselves the only estimate of terrain traversability available for these recordings. We therefore verify them in three ways, none of which uses an external reference.

First, we inspect the labels qualitatively, next to the traversed path they were computed from. The labels look plausible, in that lower values are mostly correlated with terrain that visibly impedes the robot, but they seem relatively noisy between consecutive timesteps, despite the smoothing of the RHE.

Second, we run the estimator on synthetic scenarios with known (μ, ν) and check whether it recovers them. The scenarios cover constant terrain, terrain transitions, straight-only and turn-only motion, a full stop, and a realistic mission profile modeled after a mission in the GrandTour dataset. We also apply random noise to the odometry. The results are shown in Appendix A. The estimator recovers both parameters wherever the corresponding command excites them, and falls back to the prior where it does not, which is the expected behavior. It is worth noting that the synthetic trajectories are simulated with the same kinematic model that the estimator fits. This check therefore only covers the optimization and its tolerance to odometry noise, but does not validate the labeling concept itself.

Third, we tune the RHE hyperparameters (Appendix A). We use differential evolution, a derivative-free evolutionary algorithm, since one objective evaluation runs the entire estimator over a scenario and hence provides no gradient, and since the search space is small and bounded. The criterion is the mean absolute error of (μ, ν) against the known parameters of the realistic mission scenario. This criterion is the weakest link of the three: It is evaluated on synthetic data produced by the unicycle model, so it selects weights that are self-consistent. It is not certain whether this translates to good performance on real terrain.

The three checks constrain the estimator against itself, and therefore mainly rule out implementation errors. We revisit their reliability in Chapter 9, where the human pairwise labels provide the first comparison against a different label source.

7 Training the WayFASTER Estimator on the GrandTour Dataset

As a baseline for the later experiments and the error analysis, we train a network on GrandTour with an architecture and hyperparameters close to WayFASTER's. This chapter describes that training setup, the criteria and metrics we evaluate the estimator with (Section 7.2), and the results (Section 7.3).

7.1 Implementation

Adapting the WayFASTER training pipeline to the GrandTour dataset and the ANYmal robot required some changes to the original implementation. The most important differences are listed here.

Unobservable segments We multiply the loss weight of every labeled state by the observability score we added to the RHE labels (Chapter 6). This removes standstill segments from the loss since μ cannot be identified from a robot that is not slipping.

Linear traversability only The original WayFASTER predicts both linear (μ) and angular (ν) traversability per cell. Since the yaw-rate command in GrandTour is almost always zero, we cannot meaningfully supervise ν and therefore only predict μ.

Temporal sampling WayFASTER logs at 2 Hz and stacks six consecutive frames, which span 2.5 s of driving. The GrandTour camera runs at 10 Hz, so six consecutive frames cover 0.5 s and the robot barely moves between the first and the last. We therefore take every fifth frame, which restores both the 0.5 s spacing and the 2.5 s span.

Input resolution The input is 480×360 instead of 320×180 since the Alphasense camera is a 4:3 sensor and the encoder needs a resolution divisible by 8.

7.1.1 Verification and Bug Fixes

To minimize the risk of implementation error, we verified that our adaptation of the WayFASTER pipeline reproduces the training behavior of the original implementation on the original WayFASTER dataset. We also spent considerable time verifying the individual stages of the training pipeline. This surfaced several errors in our own code (especially concerning quaternion conventions and coordinate system transformations), and it also revealed the following issues in the original WayFASTER training code.

First, BEV cells that are not seen in any input frame are not masked out in the loss, so the network is partly trained to predict labels for which it has no input. Second, the offset of the robot's base frame from the ground is assumed to be zero, which is roughly correct for WayFASTER's wheeled platform, but ANYmal's body sits about 0.55 m above the ground, so every cell is mapped to the wrong image pixels. Third, the pipeline uses cosine annealing as a learning rate schedule, but the period is set to 10 epochs while training lasts for 20, so the learning rate rises back towards its maximum halfway through training. Fourth, WayFASTER's data loader mixed up extrinsic and intrinsic Euler angle conventions, misorienting the camera on sloped terrain by an average of 8.0° and up to 68°. This was no problem for the WayFASTER dataset, which contains only flat terrain, but it is a major issue for GrandTour.

7.1.2 Software and Hardware

Training runs on PyTorch 2.12 with PyTorch Lightning 2.6 under Python 3.14, on a single NVIDIA RTX 4090 with 24 GB of memory.

7.2 Estimator Evaluation

The WayFASTER estimator outputs a per-cell prediction of the linear traversability μ. This section states the criteria that a useful prediction has to satisfy, the metrics with which we measure them, and the procedure that selects the reported checkpoint.

7.2.1 Evaluation Criteria

We chose our metrics to reflect the following requirements on the estimator's predictions, which we derived from the navigation task and the label distribution. Discrimination, calibration, and specificity are commonly used in the literature [46], [47], while minority fidelity follows from the imbalance of our label distribution.

Discrimination The estimator ranks hazardous terrain below safe terrain, independently of the magnitude of the predicted values [48].

Calibration Predictions track the target in magnitude, not only in ordering [49]. The output spans the label range rather than being compressed towards the label mean, so hazardous terrain is predicted low. The degenerate case of a constant output is ruled out.

Specificity The estimator rarely flags crossable terrain as risky.

Minority fidelity The estimator stays accurate on the rare low-μ terrain, which the $\sim 98\%$ high-μ majority would otherwise hide in an average [50].

7.2.2 Reported Metrics

We compute every metric per mission and then average over the missions. Therefore, a single long recording cannot dominate the result, and a level difference between recordings cannot inflate it. We score every archived checkpoint in this way, restricted to the cells that carry loss weight.

Metric	Definition	Criterion covered	Interpretable range $[\text{chance}, \text{ideal}]$
Calibration slope	Bin μ into ten equal-width bins, take the mean prediction per bin, and fit a line through the resulting $(\text{bin center}, \text{mean prediction})$ points. The confusion matrices visualize this line in orange.	Calibration	$[0, 1]$
AUROC $\mu < 0.5$ vs. rest	Treat the task as a binary classification: A cell with $\mu < 0.5$ is risky and has to be routed around. A cell with $\mu \geq 0.5$ is crossable. The AUROC is the probability that a random risky cell receives a lower prediction than a random crossable one.	Discrimination	$[0.5, 1]$
± 1-bin recall	Bin μ into ten bins, count a prediction as correct if it falls into the label bin or into one of its two neighbors, and average the recall over the label bins [51].	Minority fidelity	$[0, 1]$
False alarm rate (FPR)	Share of the crossable cells ($\mu \geq 0.5$) that are predicted to be risky.	Specificity	$[1, 0]$

Table 2: The four reported metrics. Each of them is computed per mission and averaged over the missions.

The four metrics are complementary since each of them is blind to a different defect: The calibration slope is fit on per-bin means, so it reflects neither how large the variance for each bin is nor how well the individual predictions are ranked within a bin. AUROC captures that ranking, but it is invariant under any monotonic rescaling of the predictions. Hence, it cannot detect a collapse towards the label mean as long as the ordering is preserved. The ± 1-bin recall tolerates any error inside its three-bin window. None of these three metrics penalizes over-caution, which is what the false alarm rate adds.

7.2.3 Checkpoint Selection and Training Monitor

In our runs, high calibration is transient: The calibration slope peaks within the first few epochs and decreases afterwards. A common practice is to do early stopping on the validation loss, but we found that the loss is not a good proxy for calibration. Since calibration is our most important quality criterion, we use this metric as the checkpoint monitor (i.e., for checkpoint selection and early stopping), and report the values of the checkpoint with the highest calibration slope.

7.3 Training Results

Here, we present the training results of the WayFASTER architecture of Section 5.1, trained on the train/validation split presented in Section 4.2 with the adaptations and bug fixes of Section 7.1, and evaluated with the metrics of Section 7.2. Of the runs that stay close to this setup, we report only the best ones, which nevertheless fall short of satisfactory performance. Runs that deviate more strongly from it are the subject of Chapter 8. We later use these results as a baseline in Chapter 8 and Chapter 9. The result is negative: The predictions contract towards the label mean over the course of training, and the calibration slope decays with them.

As Figure 9 shows, calibration peaks within the first epoch and decays from there. The AUROC score looks good at first sight. However, against a chance value of 0.5 and an ideal of 1, an estimator that a planner could act on would have to score substantially higher than the baseline does. The calibration slope is the only one of the four metrics that reads magnitudes, which is why it shows the collapse most clearly. The false alarm rate improves over the run, but only because the network predicts ever fewer cells below the risky threshold, which is the same contraction that the calibration slope penalizes. At the reported checkpoint it is far worse than a usable estimator would be since the baseline flags a larger share of the crossable terrain than the share of terrain that is risky in the first place.

The diagnosis of this collapse and the mitigations we tried against it are the subject of Chapter 8.

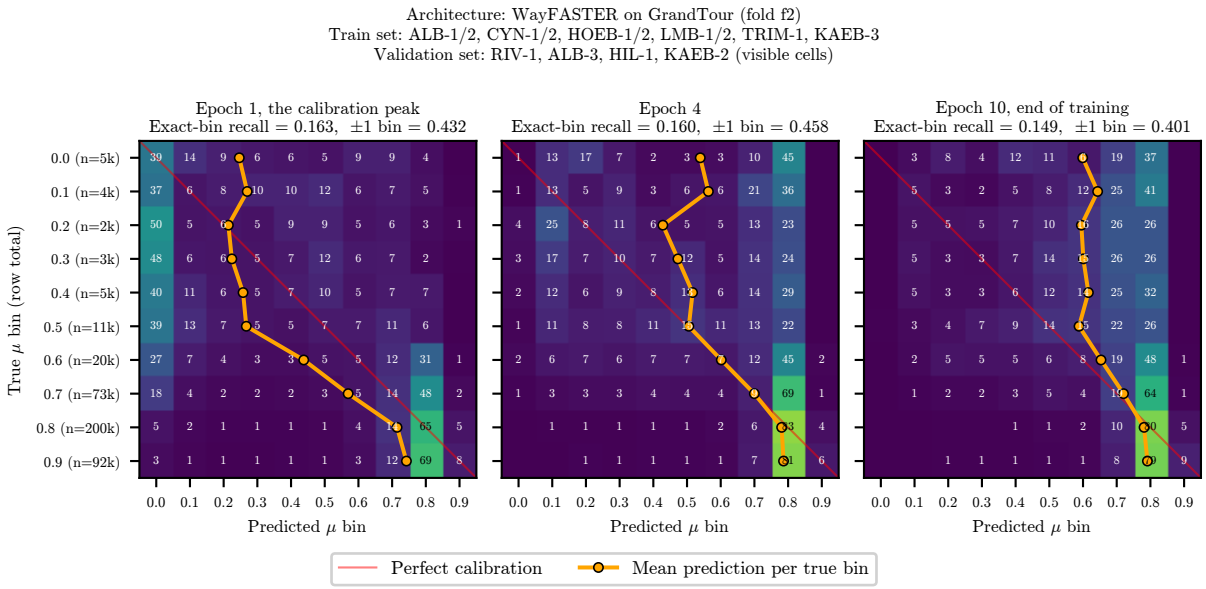


Figure 8: The baseline at its calibration peak, three epochs later, and at the end of training. Each row is one true μ bin and is normalized by its own cell count. The orange line is the mean prediction per true bin, which is the line the calibration slope is fit to. The red diagonal is where it would lie under perfect calibration. At the peak the mean prediction still varies with the label. By epoch 4 it is nearly constant below $\mu = 0.5$, and by epoch 10 the predictions of every true bin sit in the three bins around the label mean, with the two lowest predicted bins left empty.

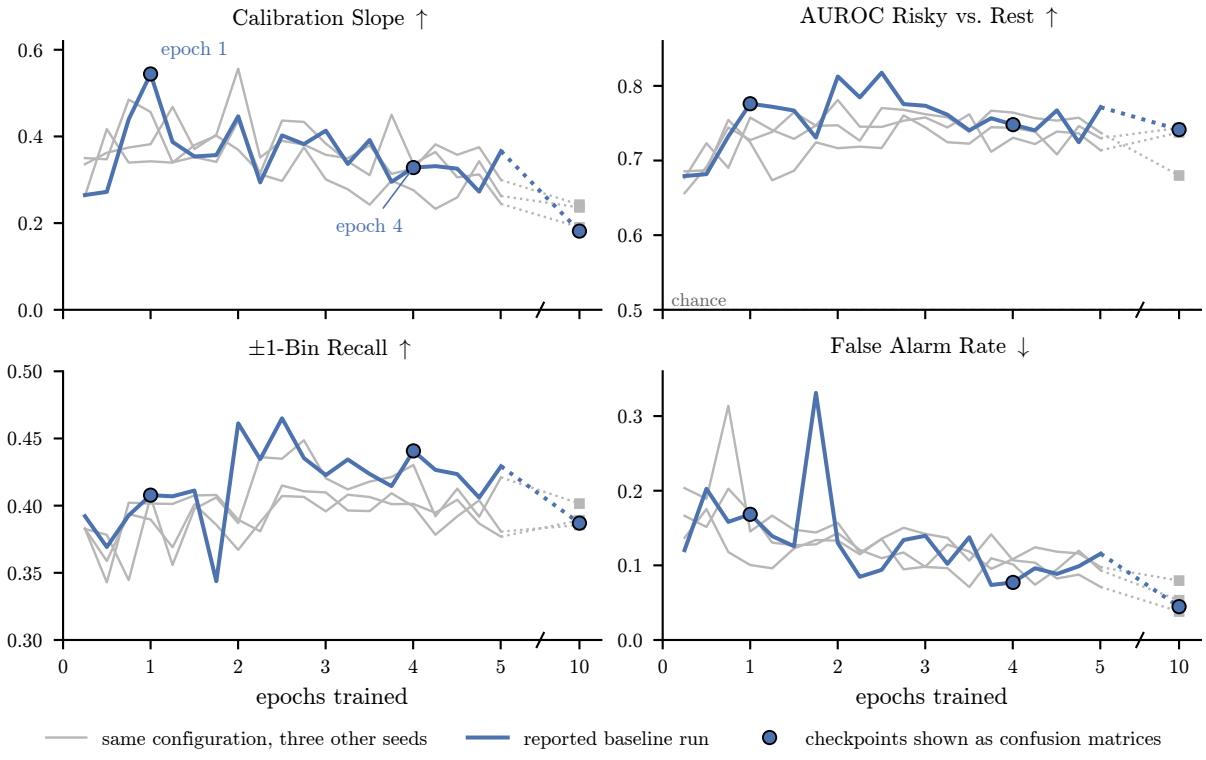


Figure 9: The four reported metrics of Table 2 over training, each averaged over the validation missions. Every vertex is one archived checkpoint, four per epoch. The axis is broken between epoch 5 and the end of training, because the metrics are almost constant after the collapse. An arrow next to a panel title marks the direction in which that metric improves. The three gray runs use the same configuration with different seeds, and fix the noise floor of Section 8.2.1. The three circled checkpoints are the ones shown in Figure 8.

8 Failure Analysis of the Trained Estimator

In Chapter 7, we observed that the WayFASTER estimator severely overfits the GrandTour-Forest dataset, with several unsuccessful attempts to mitigate that overfitting. In this chapter, we investigate the reasons for this failure.

8.1 Hypothesis for the Failure

We hypothesize that the combination of small dataset size and label sparsity makes it nearly impossible to learn patterns that generalize across recordings. We test this hypothesis in four steps:

Falsification attempt Heavily regularizing the training process from all major angles (Section 8.2).

Reference behavior Reproducing the original WayFASTER architecture on its own dataset, which is larger and has a more favorable label distribution (Section 8.3).

Collapse under a GrandTour-like label distribution Training that same architecture on its own data, only resampled to match the GrandTour distribution (Section 8.4).

Reliability of the published results Measuring the overlap between the training, validation, and field-trial environments of the WayFASTER dataset (Section 8.5).

8.2 Mitigating Overfitting and Memorization

To disprove the hypothesis, we attempt to mitigate overfitting and memorization by applying regularization techniques to the training process.

8.2.1 Noise Floor of the Evaluation

To estimate the noise floor of the evaluation, we train the same baseline configuration with $N = 4$ random seeds.[3]

Write $m(c)$ for the value that a metric m takes at a checkpoint c. Each run $i \in \{1, ..., N\}$ produces a set of checkpoints $\mathcal{C}_i$, out of which we use the one with the highest calibration slope m_{slope} (see Section 7.2):

$$c_i^* = \arg\max_{c \in \mathcal{C}_i} m_{\text{slope}}(c).$$

All four metrics of Section 7.2 are then read off this checkpoint, so every metric m ends up with one value per run, $m(c_1^*), ..., m(c_N^*)$. We define the noise floor as the standard deviation of these N values, which we denote $s_{\text{seed}}(m)$:

$$s_{\text{seed}}(m) = \text{sd}(m(c_1^*), ..., m(c_N^*)).$$

Metric	Mean	s_{seed}
Calibration slope	0.509	0.050
AUROC	0.770	0.012
$\pm$1-bin recall	0.399	0.011
False alarm rate	0.167	0.026

Table 3: Noise floor of the metrics (i.e., averaged over the four training runs with different seeds).

This allows us to express a difference between two runs as a multiple of the standard deviation $s_{\text{seed}}(m)$. We regard a difference as distinguishable from training noise only if it exceeds $2s_{\text{seed}}(m)$.

The four peaks fall in epochs 3, 0, 0, and 1, so the epoch at which calibration is best is not reproducible even when its value is. The calibration slope is the noisiest of the four metrics (see Table 3).

8.2.2 Regularization Methods

We systematically evaluate the major classes of regularization methods on the baseline configuration (shown in gray in the table), as described in Section 7.1. We also use the same training and validation sets as for the baseline. While we explored a much broader range of configurations, Table 4 summarizes the most representative methods.

[3]The runs are not fully deterministic despite the fixed seeds, because bf16 accumulation, cuDNN autotuning, and worker-dependent augmentation do not seem to use the same pseudo-random number generator.

Regularization class	Levels tested
Backbone	ResNet34, pretrained on ImageNet
	DinoV3
Backbone plasticity	Full head learning rate
	ResNet34 stages at LR $= 0.1$
	ResNet34 frozen
Decoder capacity[4]	Full decoder (12.5M trainable parameters)
	Slim decoder: `bev_compressor` as a 1×1 convolution and the deepest decoder stage dropped (12.5M to 9.9M trainable parameters)
	Slim decoder + frozen backbone (12.5M to 1.4M trainable parameters)
	Replace entire architecture: Frozen DINOv3 patch features $\rightarrow$ L2-norm $\rightarrow$ one 1×1 convolution to 20 μ bins $\rightarrow$ depth splat into BEV (15,380 trainable parameters)
Loss functions that target label sparsity	Inverse frequency weighted L1 loss
	Balanced MSE [52]
	Regression as classification over 20 μ bins, two-hot target
	Same as above, against a Gaussian target[5]
	Focal-R [50]
Weight decay (decoupled L_2 regularization)	10^{-4}
	10^{-2}
	10^{-1}
Depth head weight	0.05
	0.1
	0.5
Training data	The ten forest missions of the fold-f2 training split
	Every GrandTour mission except for the validation set (40 missions, four times the training data). The model is still evaluated on the same forest-only validation set. This does not improve the label distribution.
WayFASTER pretraining	ImageNet initialization
	Warm start from our WayFASTER reproduction instead of ImageNet initialization
	Warm Start + frozen encoder

Table 4: Regularization classes and levels tested. The baseline is our trained model from Chapter 7. Its configuration is always the first in the list (grayed out).

[4]The pretrained backbone dominates the parameter count (see Figure 4).

[5]The network predicts a binned μ distribution per pixel, and the loss is the cross-entropy between that distribution and a Gaussian around the label ($\sigma = 0.1$). For a label of $\mu = 0.6$, the ten bins centered at .05, .15, …, .95 carry the targets .000, .000, .001, .018, .129, .352, .352, .129, .018, .001. Since label trust is limited, this might be preferable to a two-hot target.

The classes address different candidate causes: Backbone plasticity and decoder capacity ask whether the network carries too many free parameters for the amount of supervision available, which the parameter count given in Chapter 5 makes plausible. The loss functions ask whether the collapse is an artifact of the objective under GrandTour's label distribution. Weight decay and the depth head weight constrain the shared encoder without touching the architecture, the latter by giving depth prediction a larger share of the gradient. The training-data experiment is the direct test of the hypothesis since it raises both the volume and the environment diversity of the training set.

8.2.3 Methods Not Included

The following seemingly obvious candidates were not included in the grid, mostly because they are redundant with other methods.

Method not included	Reason
Heavy augmentation	Experiments on the WayFASTER data showed that the number of independent events matters, rather than the amount of data alone.
Oversampling low-μ values	Adds no new events and is already addressed by WayFASTER's inverse-frequency weighting loss.
Spatial dropout	Regularizes only the smallest part of the network since the backbone contains most of the parameters.
ResNet18 encoder (6.8M $\rightarrow$ 2.1M)	Redundant with freezing the backbone.

8.2.4 Results of Regularization Experiments

Table 5 reports the best results from the regularization experiments, compared against the four-seed mean of the baseline (see Table 3). An unshaded cell is indistinguishable from the training variance of the baseline.

No version improves the peak calibration slope by more than $2s_{\text{seed}}(m_{\text{slope}})$. The 4x training set performs best in terms of calibration slope, yet the difference is approximately one standard deviation and therefore not distinguishable from seed noise. This version also appears to collapse the slowest, as visible in the calibration slope at epoch 10. This supports the *dataset-size* part of our hypothesis. Unfortunately, the 4x training set includes almost all missions, which makes a later OOD study on this configuration impossible.

8.3 Reproducing WayFASTER on Its Own Dataset

We reproduce the published results on the original WayFASTER dataset, which is openly available along with the labels. This serves as a sanity check for our training pipeline and helps us understand the failure modes that lead to the collapse observed in Section 7.3.

For comparing results, we use the confusion matrix since it is more informative than the single-number metrics of Section 7.2 when evaluating a single model only. We also show the calibration slope as an orange line as before in Section 7.3.

Arm	Epoch of Max Slope	At the Maximum-Slope Checkpoint				@ 10 Epochs
		Calibration Slope ↑	AUROC ↑	±1-Bin Recall ↑	False Alarm Rate ↓	Calibration Slope ↑
Original WayFASTER, on WayFASTER data	5	0.632	0.869	0.376	0.142	0.593
Our baseline, four-seed mean	2	0.509	0.769	0.399	0.168	0.213
Noise floor (seed spread)	± 1.4	± 0.050	± 0.012	± 0.011	± 0.026	± 0.031
Backbone						
Frozen DINOv3	1	0.544	0.770	0.397	0.158	0.264
Backbone Plasticity						
0.1x learning rate	1	0.341	0.699	0.405	0.125	0.184
Frozen	1	0.445	0.735	0.391	0.214	0.223
Decoder Capacity						
Slim decoder	1	0.397	0.715	0.374	0.178	0.218
Slim decoder + frozen backbone	1	0.298	0.679	0.397	0.159	0.225
Minimal architecture	10	0.065	0.639	0.349	**0.002**	0.065
Loss Function						
Balanced MSE	7	0.433	**0.783**	0.427	0.127	0.308
Binned CE (two-hot)	3	0.368	0.769	**0.457**	0.127	0.185
Binned CE (Gaussian)	1	0.300	0.748	0.425	0.140	0.123
Focal-R	3	0.470	0.779	0.397	0.153	0.176
Weight Decay						
0.01	2	0.461	0.762	0.394	0.147	0.213
0.1	2	0.443	0.751	0.404	0.158	0.157
Depth Weight						
0.1	2	0.460	0.764	0.387	0.149	0.244
0.5	2	0.521	**0.783**	0.367	0.166	0.311
Training Data						
4x training set size, same validation set	1	**0.562**	0.777	0.399	0.195	**0.321**
WayFASTER Pretraining						
Warm start	1	0.409	0.775	0.413	0.091	0.260
Warm start + frozen encoder	1	0.347	0.745	0.408	0.097	0.198

Cells are shaded by their distance from the baseline in noise-floor standard deviations.

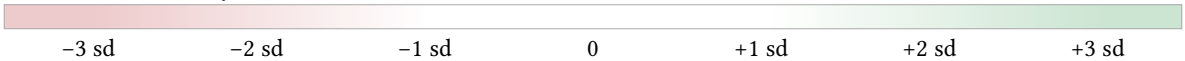


Table 5: Regularization attempts, scored against the four-seed baseline mean. Note that a low false-alarm rate can be achieved trivially by collapsing the model to predict nothing risky.

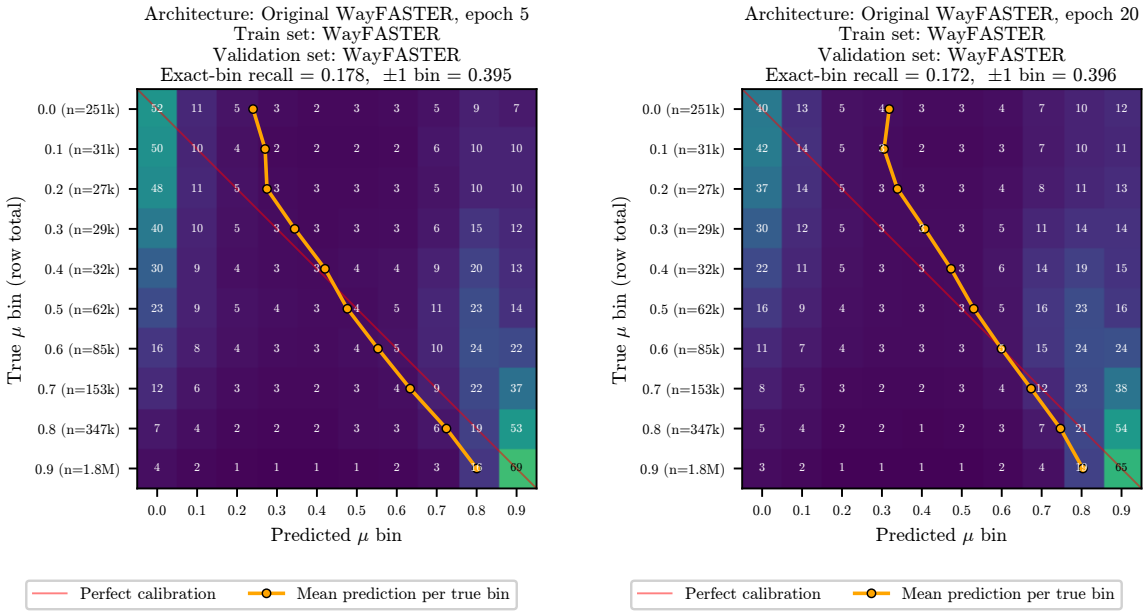


Figure 10: A reproduction of the published WayFASTER training pipeline, on the WayFASTER dataset. Left: the checkpoint of highest calibration slope. Right: the end of the run's twenty epochs.

Figure 10 suggests that the network can discern the extremes well (low vs. high traversability), because it is binning everything in the two lowest and highest bins. Its performance as a binary classifier ($\mu < 0.5$ vs. $\mu \geq 0.5$) would likely be solid, but it would struggle in the middle band. Essentially, the model is not particularly good at regressing precise μ values. The authors do not claim this, and for real-world applications in a planner, binary classification is arguably sufficient.

Calibration peaks early, at epoch 5 of 20[6], and performance does not meaningfully improve or degrade after that. This suggests that the collapse observed in Section 7.3 is not a property of the architecture or of the training code, since the same pipeline holds its calibration for twenty epochs here.

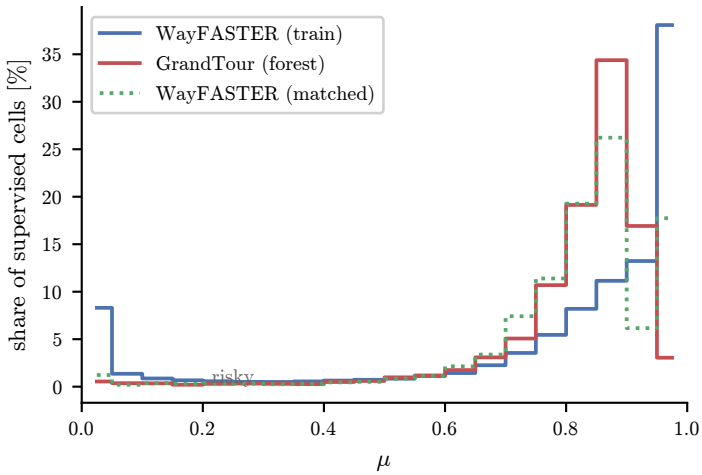
8.4 Matching WayFASTER's Labels to the GrandTour Distribution

The two datasets differ most clearly in their label distributions: The WayFASTER training set contains a considerably larger share of low-μ cells than the GrandTour forest set (Figure 11). This seems to be, for the largest part, a result of how the data was collected since the WayFASTER sequences visibly contain frequent and deliberate collisions with obstacles, which produce low-μ events.

This collection strategy was not feasible for the GrandTour dataset, because the quadruped and sensor payload are very expensive and most likely more sensitive to collisions. Therefore, the GrandTour dataset was collected with a selection bias towards traversable terrain.

To test whether this difference alone accounts for the collapse, we reshape the WayFASTER training labels to the GrandTour forest histogram. To subsample the WayFASTER dataset, we keep each cell with a probability that follows the GrandTour μ distribution (Figure 11). It is worth noting that this sampling strategy lowers the total number of supervised cells, so the experiment does not fully separate the shape of the label distribution from the amount of supervision.

[6] An epoch is a single pass over the entire training set, regardless of the training set size.



	WayFASTER all	GrandTour forest	WayFASTER matched
Recordings	47	10	47
Duration	7.07 h	1.15 h	7.07 h
Supervised cells	10.09 M	1.24 M	4.29 M
Cells with $\mu < 0.5$	14.7 %	3.8 %	4.3 %

Figure 11: The WayFASTER training labels reshaped to the GrandTour forest histogram by subsampling. This is still a more favorable scenario than the GrandTour dataset. A "cell" in this context is a single BEV patch in the world frame, so if a single patch is observed in multiple frames, it still counts as one cell.

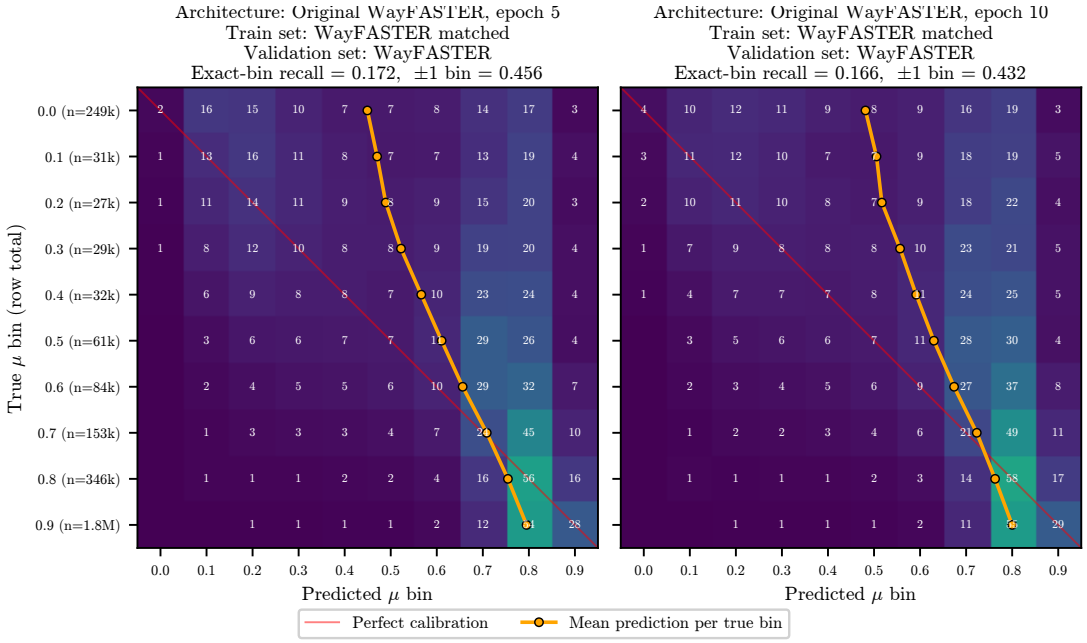


Figure 12: The published architecture trained on labels reshaped to the GrandTour forest μ distribution, scored on the full validation set at the checkpoint with the best calibration slope (epoch 5) and five epochs later.

The network collapses to almost a single bin at both the best and the last checkpoints (Figure 12). Since only the supervision changed, this supports the part of our hypothesis that assigns an important role to the label distribution.

8.5 Overlap of Train/Validation/Test Environments of WayFASTER

From manually looking through the WayFASTER dataset, it was apparent that some validation recordings were made in the same environment as some training recordings.

To quantify this, we investigated the `2023-06-28` slice, which is 2.2 h of the total 6.9 h of the WayFASTER dataset. In the slice, valid-set poses lie a median 2.5 m from the nearest train-set poses, 68% of valid poses lie within 5 m of a train pose, and 85% within 10 m.

The WayFASTER dataset ships neither lidar nor GNSS data, so we matched frames showing the same place across runs using DINOv3 image features, following the general approach of AnyLoc by N. Keetha *et al.* [53]. We then placed every run in one shared coordinate frame by fitting a rigid transform between each pair of runs and keeping the one most of the matched frames agreed on.

The paper names no location for its field trials, so we identified it from the imagery instead: The slice matches the University of Illinois at Urbana-Champaign campus. We fixed that placement by matching landmarks by hand against satellite imagery (see Figure 13).

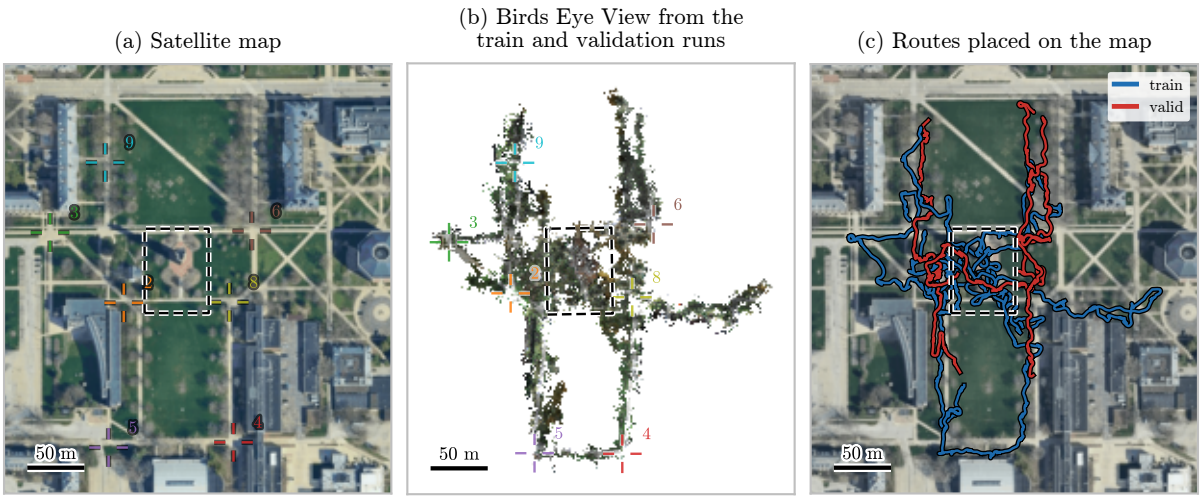


Figure 13: The `2023-06-28` slice located on satellite imagery. Numbered landmarks were matched by hand between the two panels; neither panel is warped onto the other. The gray rectangle shows the area of the field trials in Figure 14.

Applying the same matching to their published trial figure recovers the ground its panels cover, which lets us crop our own view to exactly that extent. The field trials ran on terrain the training runs had repeatedly traversed, as visualized in Figure 14.

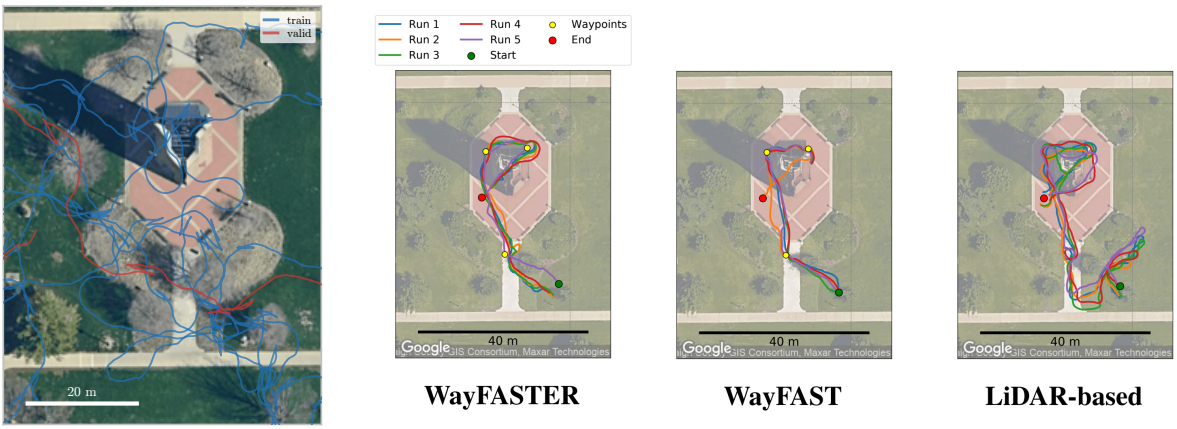


Figure 14: Left: the train and validation data, cropped. Right: the published field trials, Fig. 4 of [1]. Training data, validation data, and the field trials therefore partly cover the same ground.

The authors never claim distinct train, validation, and test sets: Their conclusion names deployment to "newly unseen environments" as future work. Further, the architecture might still exhibit generalization if the intended axis was *viewpoint* rather than *location*. The authors also conducted the *Long path navigation* experiment (Fig. 7 of [1]). There are not enough details to know whether the test path is in a new environment, and no other methods are reported for comparison.

8.6 Conclusion and Answer to RQ1

No regularization class in Section 8.2.2 improves the calibration slope by more than $2s_{\text{seed}}$, and the strongest of them, which quadruples the training data and its environment diversity, stays within one standard deviation of the baseline (Section 8.2.4). Decoder capacity, backbone plasticity, the objective, and the amount of training data therefore seem not to be the binding constraint.

The supervision is the more likely explanation. The same pipeline reproduces the published WayFASTER behavior on the original WayFASTER data (Section 8.3), and that architecture collapses on its own data once its labels are reshaped to the GrandTour μ distribution (Figure 12). A label distribution such as GrandTour's therefore appears to be sufficient to produce the collapse in a setting that otherwise trains successfully.

These results also bound what can be expected of the architecture: On the data it was designed for, it separates the extremes and assigns most of the remaining cells to the outer bins (Figure 10). It might be an optimistic bound since training, validation, and field trials partly cover the same ground, so no published result isolates generalization to a previously unseen environment (Section 8.5).

It is unlikely that the combination of WayFASTER's architecture and training setup can generalize across GrandTour recordings, with the caveat that a finite set of experiments cannot exclude a configuration we did not test. The RHE labels are biased towards traversable terrain, and they contain no events for terrain the robot has never walked. Neither property can be addressed by regularization or by more recordings of the same kind, which is why Chapter 9 changes the label scheme altogether.

9 Mitigation: Using a Relative Human Labeling Scheme

This chapter describes how we adopt a relative labeling scheme to mitigate the failure from Chapter 8.

9.1 Rationale for a Relative Labeling Scheme

Chapter 8 identifies the RHE labels on GrandTour as a binding constraint: Their μ distribution is heavily biased towards traversable terrain, and no proprioceptive label set can contain events for terrain the robot can never experience, such as trees or rivers. Neither problem is addressable by regularization or by more recordings of the same kind.

As far as we are aware, there are two classes of approaches to this situation:

Strong inductive bias Training the network with the assumption that terrain the robot has never encountered is untraversable, as done by Wild Visual Navigation (WVN) [13], among others.

Human labels An annotator can label any terrain, including terrain the robot will never experience. Most importantly, human labels are not biased towards traversable terrain. Downsides are that labeling is expensive and that labels are inferred from human body experience, which may over- or underestimate the physiological capabilities of the robot.

We did incorporate the WVN loss into our training attempts, using the Mahalanobis++ distance on DINOv3 features (Section 10.3.2) as the detector for novel terrain. While it improved the subjective results overall, it did not significantly improve the measurable performance on RHE labels.

Labeling traversability on an absolute scale is a comparatively hard task for humans [54]: The extremes of the scale, such as trees, boulders, and walls on one end and harmless ground such as trails and rock slabs on the other, are trivial to recognize. The middle of the scale, however, is much more ambiguous since the exact slip of a surface is almost impossible to judge reliably and repeatably (see Figure 15).

Relative labeling schemes avoid this problem by asking the annotator to label which of two points is more traversable, or whether they are equal. This is a pairwise ordinal comparison, which is cheap to answer and needs no absolute scale in the annotator's head.

For these reasons, we chose to adopt a relative labeling approach, more specifically the Weakly-Supervised Framework for Relative Traversability (W-RIZZ) [56]. In the following, we describe its original concept, our labeling campaign, and the results of training a network on the resulting label set.

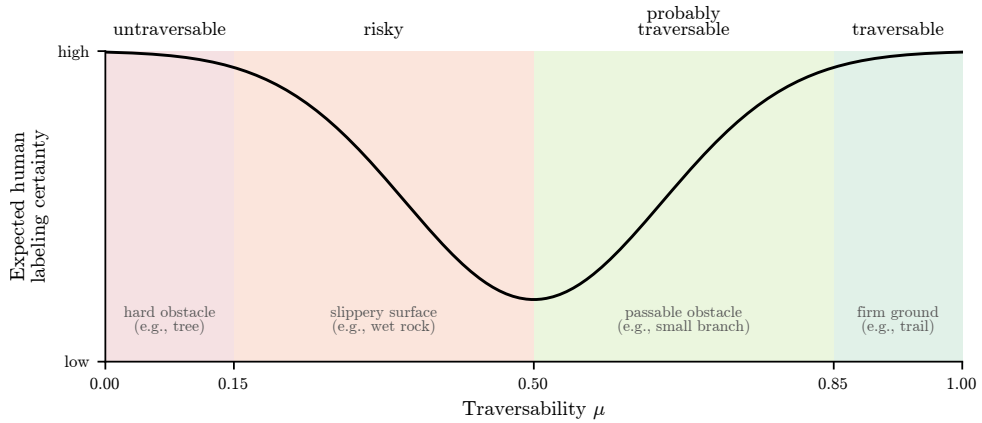


Figure 15: Expected human labeling performance. The curve is purely illustrative, and conceptually based on the bow effect [55], a finding from the field of absolute identification, which states that speed and accuracy are highest for items at the extreme ends of a stimulus continuum (e.g., the quietest/loudest tones or shortest/longest lines) and lowest for items in the middle.

9.2 W-RIZZ: A Weakly-Supervised Framework for Relative Traversability Estimation in Mobile Robotics

To acquire a different training and evaluation label set, we follow the W-RIZZ [56] approach and label pairs of points in images with a relative traversability score. The pairwise labels are then converted into a global ranking of all points, which is used to train the network.

9.2.1 W-RIZZ's Labeling Procedure

W-RIZZ [56] labels are acquired by asking a human annotator to compare two points and answer the question: *Which of these two points is more traversable?* The annotator can choose either point A, point B, or indicate that they are equal in traversability. Points can be sampled from the same image (intra-image pairs) or from different images (cross-image pairs), as illustrated in Figure 16. The latter are used to tie the score scale together across recordings. The sampling of points is biased towards the lower half of the image, which contains the terrain that the robot is going to traverse.

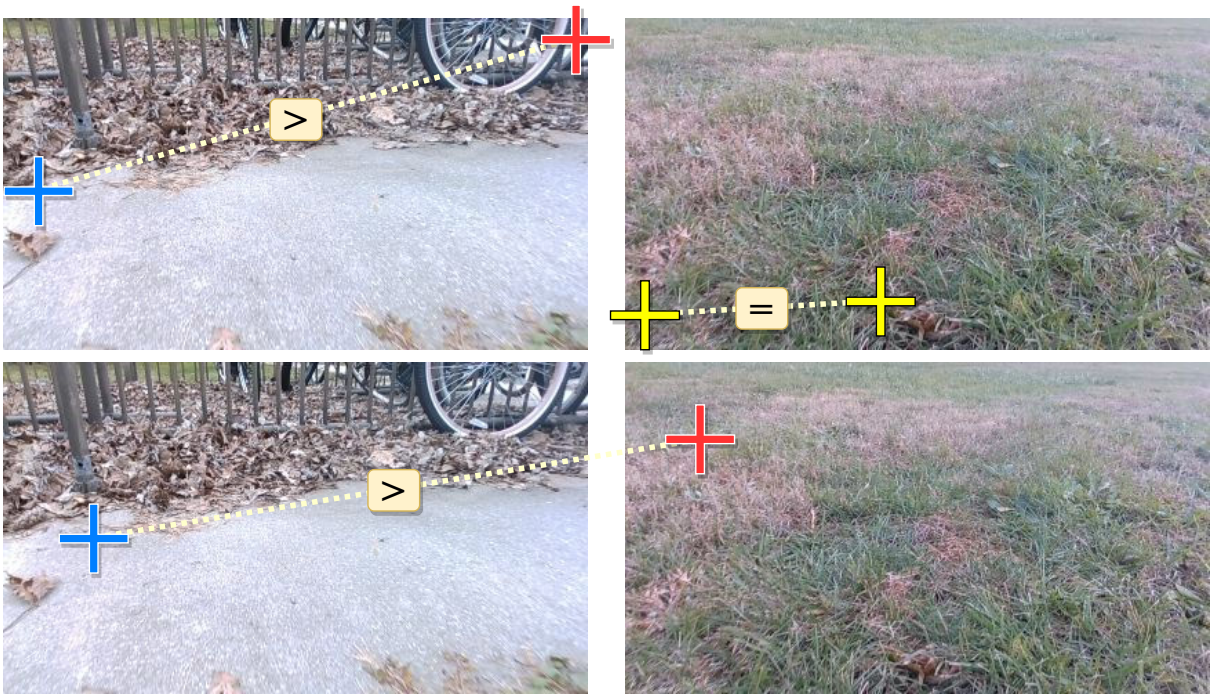


Figure 16: The W-RIZZ labeling procedure, from [56]. Two points are sampled from the same image or from different images, and a human annotator is asked to compare their traversability.

9.2.2 W-RIZZ's Loss Function

Let p_a, p_b be the network's predicted traversability at points a, b.

For better readability, we write the human labels for (a, b) as $\{l_a < l_b, l_a > l_b, l_a = l_b\}$. Note that only the relation between l_a and l_b is defined; their absolute values are not.

Let L be a hyperparameter that controls the margin. The W-RIZZ authors recommend $L = 0.5$, which we use.

Then the W-RIZZ loss function is defined as:

$$\mathcal{L}_{\text{RIZZ}}(p_a, p_b, l_a, l_b) = \begin{cases} \max\{0, L - (p_b - p_a)\}^2 & l_a < l_b \\ \max\{0, L - (p_a - p_b)\}^2 & l_a > l_b \\ (p_b - p_a)^2 & l_a = l_b \end{cases}$$

It is a squared hinge loss for inequality pairs and a squared loss for equality pairs. Note that, depending on the choice of L, the loss is non-zero for some pairs that are correctly ordered by the network. This seems to be a deliberate choice as it encourages the network to push the predictions of inequality pairs further apart, similar to a contrastive loss.

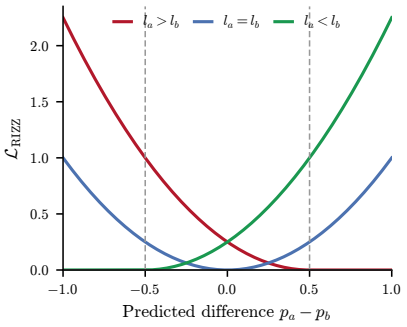


Figure 17: The W-RIZZ [56] loss as a function of the difference between predicted traversabilities. The margin at $\pm L$ is dashed.

9.2.3 Evaluation of Estimators Trained on Relative Labels

To evaluate networks trained on the human pairwise labels, we use the Human Disagreement Rate (HDR) from A. Schreiber *et al.* [56], which is an adapted version of the (confidence-) Weighted Human Disagreement Rate (WHDR) [57]. Using the HDR metric has the added benefit of maintaining comparability with the W-RIZZ paper.

HDR compares the annotator's answer t with the network's prediction $\hat{t}_\tau$.

$$t(l_a, l_b) = \begin{cases} +1 & l_a < l_b \\ -1 & l_a > l_b \\ 0 & l_a = l_b \end{cases} \qquad \hat{t}_\tau(p_a, p_b) = \begin{cases} +1 & p_b - p_a > \tau \\ -1 & p_b - p_a < -\tau \\ 0 & |p_b - p_a| \le \tau \end{cases}$$

where $p_{i,a}, p_{i,b}$ are the network's predicted traversabilities at the two points of pair i, and τ is the threshold for differences (see below for more explanation).

HDR is then the share of pairs on which the two disagree:

$$\text{HDR}_\tau = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\big(\hat{t}_\tau(p_{i,a}, p_{i,b}) \neq t_i\big)$$

More specifically, we use the *Balanced HDR*, which averages the tie rate and the inequality rate. A random predictor would achieve an HDR of 0.5 regardless of the mix of the pair set.

The threshold τ determines the range in which two predictions are considered *equal*, and therefore trades the two kinds of disagreement against each other: The larger τ, the more pairs are classified as *equal*, so equality pairs are scored as agreements more often and inequality pairs less often. At $\tau = 0$, the network practically never predicts *equal*, so every equality pair counts as a disagreement, while a large τ (e.g., $\tau = 1$) classifies every pair as *equal* and thus renders every inequality pair a disagreement. We report at $\tau = 0.25$ throughout, following A. Schreiber *et al.* [56].

9.3 The Human Pairwise Label Set

This section describes how the pairwise human labels were collected and quantifies their reliability, both internally and against the RHE labels.

9.3.1 Annotation Procedure

We deviate from W-RIZZ's original annotation procedure (Section 9.2.1) to improve annotation efficiency. The deviations are listed here.

Keyframe selection The original W-RIZZ paper samples keyframes at a fixed stride, which can lead to redundant pairs. We use a greedy k-center (cosine distance) algorithm on the DINOv3 features of the frames to select a diverse set of keyframes for annotation. This ensures that the selected frames are representative of the visual diversity in the dataset and reduces the number of redundant pairs that need to be annotated.

Geometric expansion The same patch of ground is also visible in the frames before and after it. We project the label to the surrounding frames by using the depth and odometry information, so that one answer supervises multiple points. A frame is kept only if the depth it measures there agrees with the depth of the labeled pixel (to avoid occlusions).

Segment expansion The label is extended to the SAM [16] segment for the point to maximize label coverage. For the two segments A, B and a relation $r \in \{<, =, >\}$, the following must hold:

$$\forall a \in A, b \in B: \quad a \, r \, b$$

The annotator can dynamically shrink or completely veto the segment expansions and must ensure that the above relation holds.

Point selection For pairs where both points are in the same image, the points must land in different SAM segments to maximize visual difference.

Training-eligibility gate To ensure separation between training and evaluation data, a pair may enter the loss only if both points lie in train missions. Everything else is only used for validation and evaluation.

9.3.2 Scale of the Human Label Set

A single annotator produced the entire label set consisting of 3581 answered pairs on 2695 keyframes. More details are reported in Table 6. Of the labels, 362 pairs were skipped, meaning they were not accurately answerable by the annotator.

Split	Missions	Keyframes	Same Image	Cross-Image	Answered Pairs
train	10	1387	1334	656	1990
held-out	7	526	426	223	649
OOD tiers[7]	22	1023	404	828	1232
Total	32	2704	2164	1707	3871

Table 6: The scale of the pairwise labeling effort.

[7] Evaluation only: these place the tiers on the traversability-shift axis of Figure 18 and never enter the loss.

9.3.3 Measurement of Self-Consistency

To measure the label consistency, a subset of pairs is re-asked with endpoints swapped and without the SAM segment overlay. The re-asks are answered in a later session (20 days later) since a same-session re-ask measurement could be contaminated by memory of the first answer. The re-asks are a random subset of the original pairs, with a minimum 2 s gap between them.

Over the 337 re-asked pairs, the annotator repeats their own answer on 78% of them. Of the rest, 19% are a tie against an ordering and 3% are an opposite ordering. Visual inspection of the disagreements showed that a tie against an ordering is most often a boundary call on a pair that is difficult to label precisely, while opposite orderings are clear contradictions that make the pair unusable. Another source of error is the SAM segment

This highlights an important limitation of the human labels. As shown in Figure 15, the middle of the traversability scale is ambiguous, and a human annotator is not always able to decide whether a difference is worth stating. The annotator's own uncertainty is therefore a ceiling on the HDR metric of Section 9.4, which measures how well the network reproduces the human labels.

9.3.4 Disagreement with RHE Labels

We also measure the disagreement between human and RHE labels of Section 6.1. RHE is not a ground truth to score the human answers against, since, as mentioned in Section 6.3, there was no way to verify the correctness of the RHE labels, and they are heavily biased towards high traversability values.

Of the 2349 answered ID pairs, 551 have an RHE label with observability ≥ 0.6 at both endpoints. To convert the absolute RHE labels into a relative ordering for comparison with the relative human labels, we use the same threshold, $\tau = 0.25$, as in the HDR metric of Section 9.2.3. More concretely, we call a pair of RHE labels a tie if their difference is at most $\frac{\tau}{2} = 0.125$, and an ordering if it is larger than $\tau = 0.25$. For differences between $\frac{\tau}{2}$ and τ, the pair is considered ambiguous and discarded from the comparison. This leaves 495 comparable pairs. The two label sets disagree on 28% of them, and none of the disagreements is a reversal (Table 7).

Outcome	Pairs	Share
Comparable pairs	495	100%
Agreement	359	72.5%
Disagreement	136	27.5%
RHE: *Tie*, Human: *Ordering*	127	25.7%
RHE: *Ordering*, Human: *Tie*	9	1.8%
Reversal, i.e., opposite orderings	0	0.0%

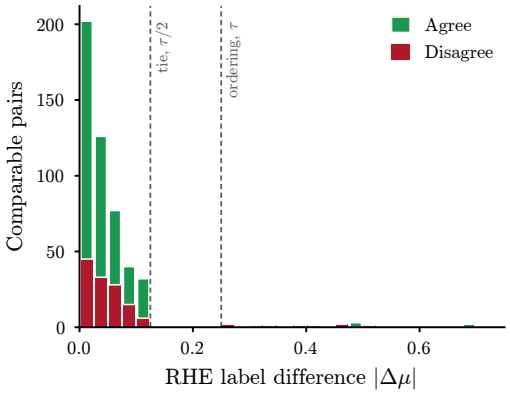


Table 7: Left: human labels against RHE labels, on the ID pairs where both exist. Right: disagreement plotted as a function of the RHE label difference $|\Delta\mu|$. Pairs between the dashed lines are considered ambiguous and discarded from the comparison.

As visible in Table 7, disagreement occurs almost only in situations where RHE labels are close to each other but the annotator calls the pair different. This highlights another limitation of the human labels. In some situations, surfaces that look different in traversability may actually be similar in terms of the experienced traversability, as measured by proprioceptive labels.

9.4 Training Results

Here, we describe the training of Section 5.1 on the human labels of Section 9.3 and score it with the HDR of Section 9.2.3.

9.4.1 Training Procedure

In the following, we describe the training procedure of the best-performing configuration.

Every answer gives us a pair of labeled segments (A, B). The naive training approach would be to sample every possible point pair $(p_a, p_b) \in A \times B$ and compute the loss on the network's predictions at all of those points, which overrepresents large segments in the loss. Instead, we average the predictions over each segment before computing the loss, which improves performance, likely because it reduces the noise in the predictions and thereby makes the loss more stable. Since ties make up 55% of the answers, we reweight the tie and inequality mass so that inequalities account for 0.75 of the loss. This strongly improves prediction performance, presumably because inequality pairs carry more information about the ordering of the label space than tie pairs do. Following W-RIZZ [56], we also use the mean teacher approach [58] to stabilize training and improve generalization.

9.4.2 Results

Table 8 compares our estimator against the best results reported in the W-RIZZ paper [56] and against their reported baseline. Note that the two upper rows do not allow a head-to-head comparison with our results: They use a different robot, a different dataset, and a pair population drawn under different sampling rules.

Origin	Architecture	Dataset	Label type	Label count	Balanced HDR $\mathrm{HDR}^{\mathrm{bal}}_{0.25} \downarrow$	Tie Error $\mathrm{HDR}^{=}_{0.25} \downarrow$	Inequality Error $\mathrm{HDR}^{\neq}_{0.25} \downarrow$
W-RIZZ (Baseline)	WAYFAST	WayFAST DS	Human (relative)	25k	0.311	0.124	0.658
W-RIZZ (Baseline)	W-Rizz TravNet	WayFAST DS	Human (relative)	25k	0.125	0.044	0.275
Ours	Ours, based on WayFASTER	GrandTour (Forest)	RHE	n/a	0.422	0.096	0.747
Ours	Ours, based on WayFASTER	GrandTour (Forest)	Human (relative) + SAM	2.5k	0.249	0.098	0.399
					0 0.5 1	0 0.5 1	0 0.5 1

Table 8: Our human-label estimator against published methods, on the three HDR metrics of Section 9.2.3. The gray rows are the baselines of A. Schreiber *et al.* [56] the orange rows are our trained network on GrandTour, each a mean over four seeds with the standard deviation as the whisker.

Supervised by the RHE labels alone (Table 8, row 3), the network reaches a balanced HDR of 0.42 ± 0.03, against the 0.5 achieved by a random predictor. The low tie error paired with the high inequality error suggests that the network collapses to a narrow output range, which favors the tie loss but fails to generalize the inequality labels, consistent with the findings of Chapter 8.

Supervised by 2.5k human pairs (Table 8, row 4), the two error types are far more balanced. Our method does not reach the performance of the W-RIZZ baseline (row 2), but it is trained on a tenth of the human label pairs. Training the W-RIZZ architecture on the GrandTour human label pairs did not yield a better HDR than our own architecture (best $\mathrm{HDR}^{\mathrm{bal}} = 0.32$), so we assume that the performance gap is mainly attributable to the difference in datasets and label budget rather than to the architecture.

For the OOD comparison, this level of performance is sufficient, since all detectors are evaluated on the same estimator. The main requirement is that the estimator learns a meaningful representation of terrain traversability instead of collapsing to a trivial or degenerate solution, and both the HDR metric and a qualitative inspection of the predictions support that this is the case.

10 Application of OOD Detection Methods

This chapter asks which post-hoc OOD detection methods flag terrain the estimator was not trained on. We first group the OOD recordings by visual and by traversability novelty, fix the evaluation metrics, and introduce the five methods under comparison. The results show that no single detector covers all four novelty regimes, and that the failure modes are complementary: detectors of visual novelty miss the silent hazards, and vice versa. We therefore close the chapter with an ensemble of methods with complementary failure modes.

10.1 OOD Tiers

To evaluate OOD detection methods, we classify the OOD recordings along the following two axes of novelty, measured relative to the ID (forest) recordings.

Covariate (Visual) Novelty A change in the appearance of the terrain, but not necessarily in its traversability. An example of this would be a paved street, which is visually very different from the ID forest class the robot was trained on, but is still trivially traversable.

Semantic (Traversability) Novelty A change in traversability, but not necessarily in terrain class. In our situation, an example of a semantic-only shift would be a forest environment with higher grass than the training set.

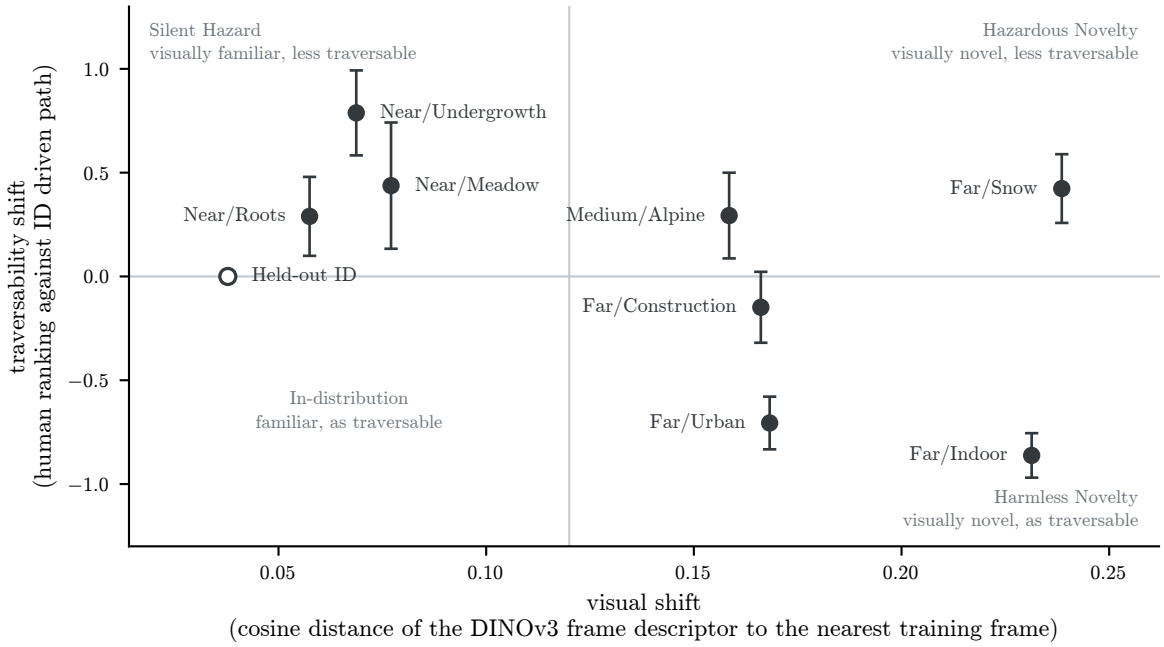
This distinction follows the distinction between covariate and semantic shift in the OOD literature [7], [8]. Sensitivity to appearance and sensitivity to a changed input-output relation are known to trade off against each other in OOD detection [12], so a detector that scores well on the first axis does not automatically perform well on the second. An ideal detector should be able to separate both types of novelty, and we therefore evaluate the methods on both axes.

Table 9 lists the tiers and the recordings they consist of.

	Tier	Content	Missions	Visual Shift	Trav. Shift
	**Held-out ID**	Forest and non-alpine hiking path never trained on	ALB-3 HIL-1 (forest) KAEB-2 RIV-1 (forest)	None	None
	**Near/Roots**	Wooded hiking path over large roots and rock	ROOT-1	Minimal, same environment type	Large, explicitly less traversable
	**Near/Undergrowth**	Dense entangling undergrowth	KAEB-1 (undergrowth)	Small to moderate, forest vegetation but cluttered geometry	Large, dense vegetation impedes traversal
	**Medium/Alpine**	Alpine gravel paths, Eiger and Pilatus	EIG-1 (natural) EIG-2 (natural) PIL-2 (natural)	Moderate to large, openness and lighting	Moderate, rocky and scree
	**Far/Construction**	Construction site and gravel pit, outdoors	CON-3 (outdoor) CON-4 HEAP-1 (outdoor)	Large, man-made, machinery, mixed materials	Moderate to large, obstacles and rubble
	**Far/Indoor**	Manufactured floor, no sky and no vegetation	ETH-2 HAUS-1 LEE-1 LEICA-2 (indoor)	Very large, artificial light, no sky, confined	Small to moderate, flat floors but walls and clutter
	**Far/Urban**	Paved campus and street, buildings and traffic	ETH-1 ETH-3 GRI-1	Large, facades and planar structure	Small, flat pavement is trivially traversable
	**Far/Snow**	Packed alpine snow	SNOW-1 SNOW-2 SNOW-3	Large, texture-poor white surface	Moderate, walkable but slippery

Table 9: The OOD tiers we selected, with one representative frame and the GrandTour missions each tier is drawn from. The two shift columns state where we expect a tier to sit; this is measured in Figure 18.

To place the missions in the quadrants, we measure both shifts. The **visual shift** of a tier is the mean cosine distance between its pooled DINOv3 frame descriptor and the closest training frame. For quantifying the **traversability shift** of missions, we labeled a small representative set of keyframes. More concretely, we pick 120 keyframes per tier by greedy k-center on their DINOv3 features, so that the labels cover the appearance range of a tier instead of its most frequent view. To ensure that the chosen labels mainly represent geometrically traversable terrain, we only sample points at most 0.2 m above the ground. We also mainly pick cross pairs between tiers to ensure that the labels measure the difference between the tiers. The border between familiar and novel appearance is chosen as the largest visual shift among all held-out ID frames. Figure 18 shows the outcome.



617 anchored pairs, counting labeled points at or below 0.2 m over ground. Bars are 95 % intervals of the mean.
Vertical line: the visual-shift border at 0.12.

Figure 18: The OOD tiers with both shift axes measured, held-out ID at the origin. The vertical line is the border between familiar and novel appearance, and the corner labels name the four regimes it and the horizontal line cut out.

Most of Far/Construction is packed ground with only a few novel obstacles, which is why it sits on the border of the traversability-shift axis.

An ideal OOD detector should behave as follows on the four quadrants of Figure 18: *Held-out ID* terrain should not be flagged, since it is the terrain the estimator was trained on. *Silent hazard* and *novel hazard* missions should be flagged as OOD since they carry a risk that the traversability estimator was not trained to detect. Not flagging *harmless novelty* missions poses no safety risk. Still, an OOD signal is useful for these missions since a visually novel environment can be a source of error for the estimator. Consider the indoor case: Most of the floor is traversable, but the estimator might misclassify a wall as traversable floor.

10.2 OOD Evaluation Metrics

To evaluate the performance of out-of-distribution (OOD) detection methods in the traversability estimation domain, we treat OOD detection as a binary classification problem where samples are classified as either in-distribution (ID) or out-of-distribution (OOD). The negative class is always the held-out ID windows (ALB-3, HIL-1, KAEB-2, RIV-1). The positive class is the windows of the currently evaluated tier. Sliding a threshold over the score then traces the ROC curve, the true positive rate being the share of that tier's windows flagged and the false positive rate the share of held-out ID windows flagged.

The OOD-detection literature shows an overwhelming use of three metrics for evaluating OOD detection performance: AUROC, AUPR, and FPR95, where AUROC is often the primary metric [7], [9], [10]. We focus on AUROC and FPR95: This combination gives a solid assessment of the methods' performance and comparability with other work. G. Humblot-Renaux *et al.* [59] notes that these standard metrics are blind to the level of separation between ID and OOD scores, which is why we also investigate the histograms of OOD scores in Section 10.5.

10.2.1 AUROC

AUROC is defined in Section 2.4. It measures the ability of a model to distinguish between ID and OOD samples across various decision thresholds. This threshold-free property is particularly useful in our context, where the cost of false positives and false negatives can vary significantly depending on the operational scenario.

10.2.2 False Positive Rate at 95% True Positive Rate (FPR95)

The FPR95 metric measures the proportion of ID samples misclassified as OOD when the detection threshold is set such that 95% of true OOD samples are correctly identified. Formally:

$$\mathrm{FPR95} = \mathrm{FPR}(\tau_{95})$$

where τ_{95} is the threshold satisfying $\mathrm{TPR}(\tau_{95}) = 0.95$. This metric is relevant for safety-critical applications: Our main objective is to keep the *false negative rate* low (so we do not overlook OOD terrain), which corresponds to achieving a high *true positive rate* (catching 95% of dangerous OOD terrain). A secondary objective is to reduce the number of false alarms (false positive rate). Lower FPR95 values indicate better performance, with 0% being optimal.

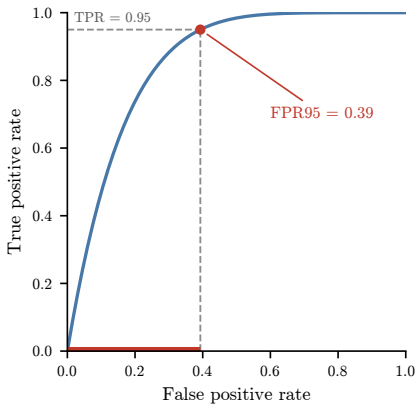


Figure 19: FPR95 read off the same kind of ROC curve as Figure 2: Fix the threshold where the curve crosses a true positive rate of 0.95, then read off the false positive rate there.

10.3 Methods Under Comparison

Section 3.2 introduces the method families surveyed here. We restrict the comparison to post-hoc methods since they require no retraining of the estimator, whose training procedure was relatively unstable.

Two methods were considered but left out. Evidential regression [33] strongly destabilized the already unstable training of the RHE-based architecture, so we avoid adding further training-time components. It is also not directly applicable to the relative-label architecture. GradNorm scores a sample by the norm of the gradient of the KL divergence between a softmax output and the uniform distribution, which has no counterpart in the two-channel sigmoid regression head used here. It would also not have widened the method set significantly, as the gradient norm is dominated by its feature-norm term and therefore orders samples much like the feature-distance scores already included.

10.3.1 RQ2: Mahalanobis++ Distance on Network Features

The Mahalanobis distance measures how far a sample lies from a distribution. It is the Euclidean distance after whitening with the covariance Σ:

$$d_{\text{Mahalanobis}}(u, m, \Sigma) = \sqrt{(u-m)^\top \Sigma^{-1} (u-m)}$$

A step along a direction of high variance increases the distance less than a step across it (see Figure 20).

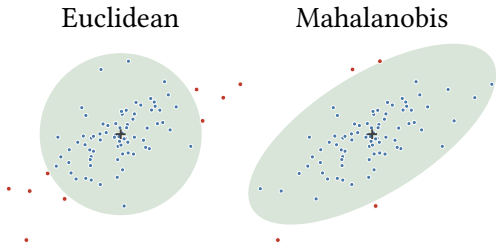


Figure 20: A stretched feature distribution under both distances. Blue points are in-distribution samples, red are outliers, the cross is the mean, and the shaded region is what each rule calls in-distribution.

K. Lee *et al.* [26] first used the distance for OOD detection. Mahalanobis++ [60] additionally ℓ_2 -normalizes the features before the Gaussian is fitted. This removes the feature-norm term, which otherwise dominates the distance and makes the fitted distribution heavy-tailed.

10.3.1.0.1 Feature Spaces

The distance is normally computed on penultimate-layer features, which are a single vector per sample. Ours are not: The penultimate layer is a 100×100 grid of 64 channels (Figure 21).

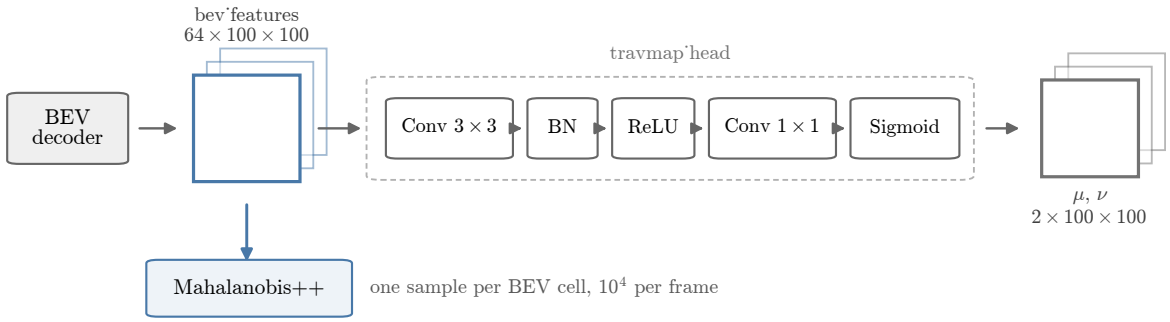


Figure 21: The layer at which Logit-Mahalanobis++ taps the network, giving one 64-dimensional vector per BEV cell. The remaining head collapses those 64 channels onto the two output channels, so this is the last point at which a cell carries more than its predicted traversability.

We fit the distance in two feature spaces and report the two as separate detectors. *Logit-Mahalanobis++* uses the BEV features above, one sample per BEV cell. *Encoder-Mahalanobis++* uses the per-pixel output of the image encoder, one stage earlier, before it is projected into the frustum: a sample is then one image pixel, and nothing else about the method changes. Those features carry no depth, since the

depth distribution is the other half of the encoder output and the measured point cloud enters only after the projection. Comparing the two shows how much of the novelty signal survives the projection and the layers that follow it.

10.3.1.0.2 Application to the Traversability Estimator

Algorithm 1 formalizes how we apply Mahalanobis++ to the WayFASTER estimator. Figure 22 shows the procedure schematically.

We treat every BEV cell as its own sample. Averaging the grid into one vector would dilute a local anomaly over 10^4 cells[8]. Cells that are not visible in any input frame are dropped, as their input features are undefined and cannot be scored. The remaining cells are normalized and subsampled at 512 per window into a buffer. This number is a compromise between the need for a representative sample and the need to keep the buffer small enough for storage. With roughly 7000 observed cells per window, 512 random samples provide a reliable estimate of the true median while keeping the cached block small.

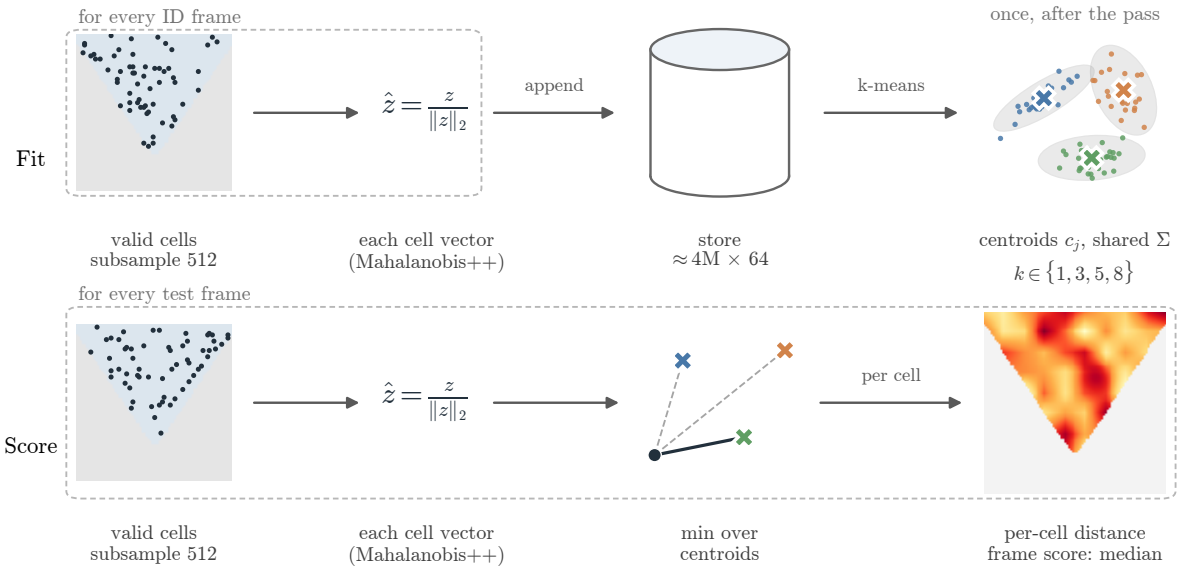


Figure 22: Mahalanobis++ over BEV cells. Fit (top): 512 observed cells per ID frame are ℓ_2-normalized and pooled into one store, over which k-means gives the centroids and their covariances. Score (bottom): every cell of a test frame takes the distance to its nearest centroid, and the median over cells is the frame score. Mahalanobis++ normally forms one distribution per class, which k-means replaces here since the task is a regression.

[8]Only ~70% of the grid is visible in any input frame, so the effective number of cells is smaller.

Fit (ID missions)
  for each ID frame **do**
    $Z \leftarrow$ `bev_features` ▷ $z_i \in \mathbb{R}^{64}$ *per BEV cell (in the 100x100 grid)*
    drop every z_i no input frame observes
    $\hat{z}_i \leftarrow z_i / \|z_i\|_2$ ▷ *Mahalanobis++ [60]*
    append 512 uniformly drawn $\hat{z}_i$ to the buffer B
  end for
  $m_1, \ldots, m_k \leftarrow$ k-means over B ▷ *terrain is probably not unimodal*
  for each centroid m_j **do**
    $B_j \leftarrow \{\hat{z} \in B : m_j \text{ is the nearest centroid}\}$
    $\Sigma_j \leftarrow \mathrm{cov}(B_j)$ ▷ *Empirical covariance*
    $\mathrm{cov}(B_j) \mathrel{+}= 10^{-6} I$ ▷ *avoid division by 0; | ℓ_2 normalization leads to zero spread along one direction*
  end for
Score (any frame)
  same mask, same ℓ_2-normalization ▷ *same 512-cell sample*
  $d_i \leftarrow \min_j d_{\text{Mahalanobis}}(\hat{z}_i, m_j, \Sigma_j)$ ▷ *nearest mode*
  frame score $\leftarrow$ median of $\{d_i\}$ ▷ *the mean dilutes, the max is noisy*

Algorithm 1: Mahalanobis++ over BEV cells, with $d_{\text{Mahalanobis}}(u, m, \Sigma) = \sqrt{(u-m)^\top \Sigma^{-1} (u-m)}$.

Most other methods using Mahalanobis for OOD detection keep one Gaussian per label class [60]. We could replicate this by categorizing our μ values into n bins and keeping one Gaussian per bin. However, we hypothesize that the same μ value does not imply that cells are similar in feature space. For example, asphalt and packed dirt may be similarly traversable, but could be represented very differently in feature space. For this reason, we use k-means to find clusters. We set $k = 3$. Sweeping k over $\{1, 3, 5, 8\}$ leaves the ranking of the detectors unchanged; only Encoder-Mahalanobis++ gains slightly with larger k.

We tested three distance measures in this setup: the Mahalanobis distance, the Euclidean distance, and the cosine distance. The cosine and Euclidean distances performed similarly, but both were outperformed by the Mahalanobis distance, so we report only its results here.

We also considered using a segmentation method like SLIC to group BEV cells into superpixels and then compute the Mahalanobis++ distance on the averaged features of these segments. However, we found that this approach was not suitable for our task. The Mahalanobis distance is sensitive to the variance of the features, and averaging over segments can erase important frequency structures in the feature space. Additionally, SLIC groups cells based on color similarity, which may not correspond to similarity in the learned feature space.

Every observed cell is scored by its distance to the nearest mode. The window score is the median over those cells. We also tried the mean of the top 10% of OOD scores but found it to be more sensitive to outliers and less stable across different OOD tiers. The median provides a more robust measure of the overall OOD level in the window.

10.3.2 RQ3: Mahalanobis++ Distance on DINO Features

As already mentioned in Section 3.2, OOD detection methods based on large pretrained models rely on the assumption that the features learned by large models on large datasets are more general and transferable than those learned by smaller models on smaller datasets.

DINOv3 [61] is a recent (2025) addition to the DINO family of self-supervised vision transformers. We extract features with its `dinov3-vitb16-pretrain-lvd1689m` architecture (frozen) and apply the Mahalanobis++ distance as in Section 10.3.1. Its 768-dimensional feature vectors have the advantage of not requiring dimensionality reduction.

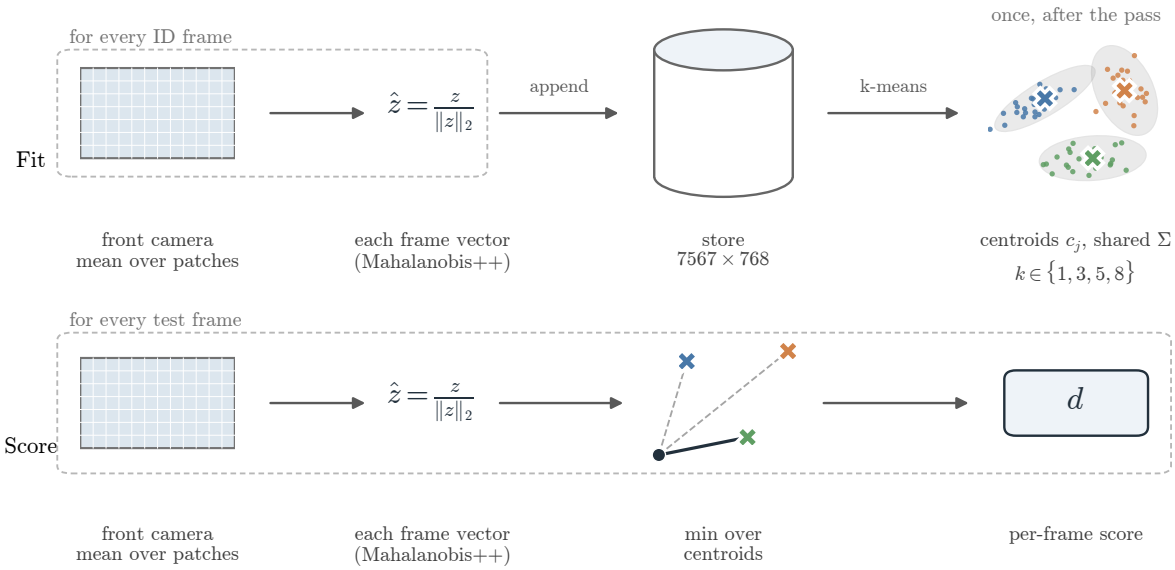


Figure 23: The same fit and score on frozen DINOv3 features. One vector per frame, the mean over the front-camera patches, replaces the 512 cells, so the store holds one row per frame and the score is that frame's distance to the nearest centroid.

10.3.3 RQ4: Depth Head Predictive Entropy

The depth head of the upper branch of the WayFASTER architecture outputs a probability distribution over depth bins. The predictive entropy of this distribution can be used as an uncertainty measure, with higher entropy indicating greater uncertainty in the depth prediction. The hypothesis is that unfamiliar terrain will lead to higher predictive entropy, as the model is less confident in its depth predictions for OOD inputs.

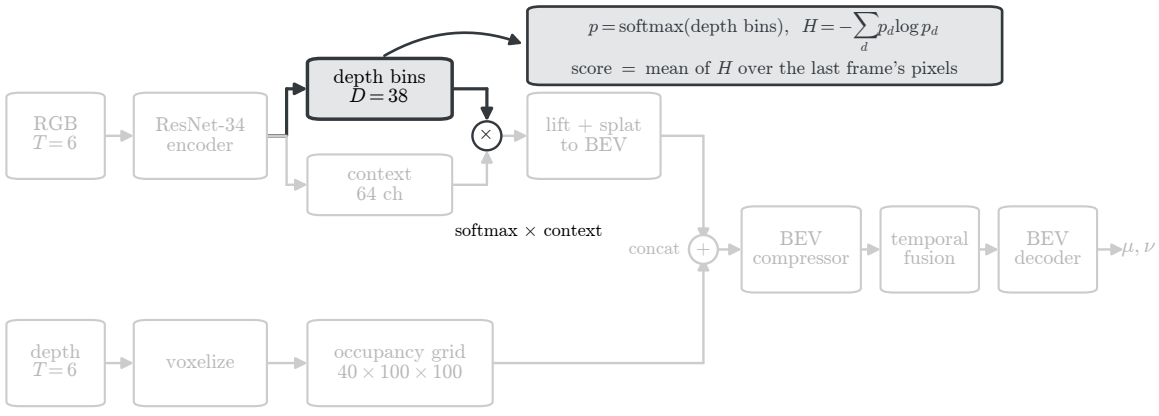


Figure 24: The depth-entropy score on a simplified diagram of the WayFASTER architecture. The depth head emits a distribution over the 38 depth bins, and the score is the mean predictive entropy H of that distribution over the last frame's pixels.

10.3.4 RQ5: Disagreement Under Viewpoint Change

Since we have relatively accurate odometry information, we can compare the same coordinate in world space across multiple frames. If the predicted traversability for the same world coordinate varies more than the ID baseline across different viewpoints, it may indicate that the model is uncertain about that region, which could be a sign of OOD terrain.

Every window is anchored on its own last input frame, so the cell centers of window i have world coordinates and the odometry gives the rigid transform $T_{i\to j}$ into any later window j of the same mission. We take the next 24 windows at stride one as probes and read each of their traversability maps at those coordinates (Figure 25). Counting the anchor itself, a cell center c seen by n_c of these viewpoints is predicted n_c times, and the cell statistic is the largest gap between any two of the predictions:

$$\Delta\mu(c) = \max_j \mu_j\big(T_{i\to j}c\big) - \min_j \mu_j\big(T_{i\to j}c\big)$$

Cells with $n_c < 6$ are dropped, and the window score is the median of $\Delta\mu$ over the remaining ones, undefined when fewer than 100 of them remain. A cell only has to move once to be counted, while a single bad pose cannot move the median over the cells. Probes run forward only: cells close to the anchor barely change between viewpoints, so the signal is in the far ones, predicted before the camera can resolve them. Neither labels nor a second model are involved, only the estimator's own output from several poses.

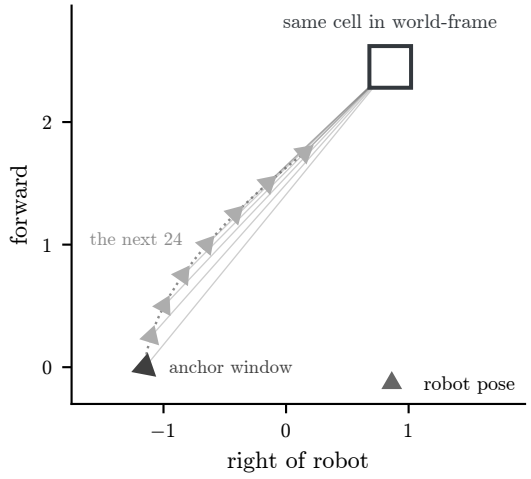


Figure 25: Disagreement under viewpoint change: every window whose footprint covers a cell predicts that cell once, and the cell statistic is the spread over those predictions. Schematic, not a measurement.

10.3.5 RQ6: Modality Disagreement Between RGB and RGB+Depth

Because the network is trained with a 30% depth modality dropout, the RGB-only inference path should also be capable of predicting traversability. This method effectively uses the different input modalities of a single model as an efficient pseudo-ensemble. The score is the absolute difference between the mean traversability predicted by the two paths (Figure 26).

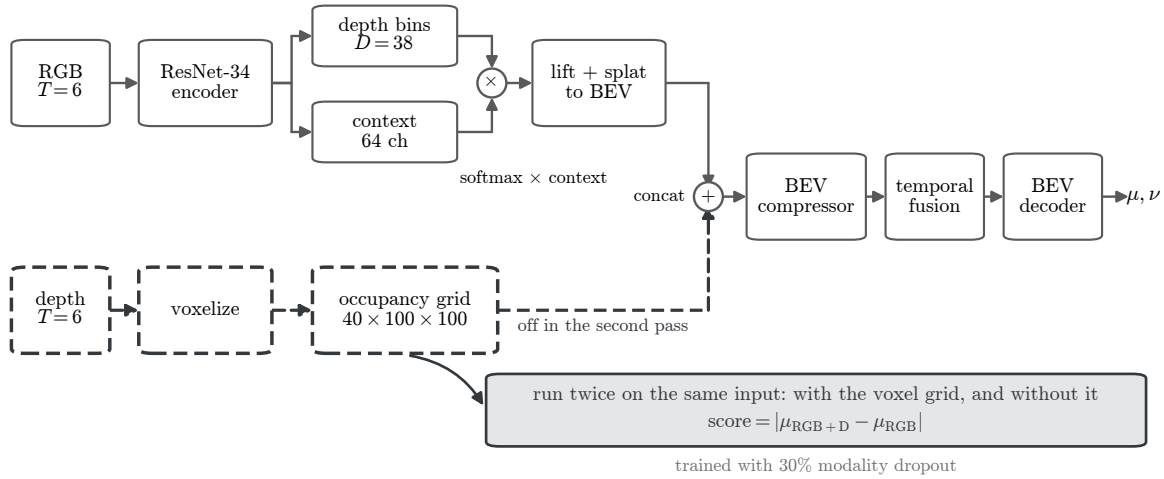


Figure 26: The modality-disagreement score. The network is run twice on the same input, once with both branches and once with the depth branch removed (dashed), and the score is the absolute difference of the two μ predictions.

10.4 Results of the OOD Detection Methods

In this section, we present the results of the OOD detection methods. First, we establish a baseline by evaluating the detectors on held-out in-distribution recordings (Section 10.4.1), then we report the results of the detectors on the OOD tiers (Section 10.4.2).

10.4.1 Performance on the Held-Out ID Set

Ideally, an OOD detector should rate all ID recordings with a low OOD score. Failing to do so indicates overfitting to the training recordings.

To quantify this, we evaluate all six unique pairs from our four held-out ID recordings (ALB-3, HIL-1, KAEB-2, RIV-1-forest). We compute the AUROC for each pair and average them into $\mathrm{AUROC}_{\mathrm{ID}}$, a per-detector baseline score. We choose pairwise held-out evaluation over comparing training vs. held-out ID recordings, as the latter task is trivially easy for the detectors.

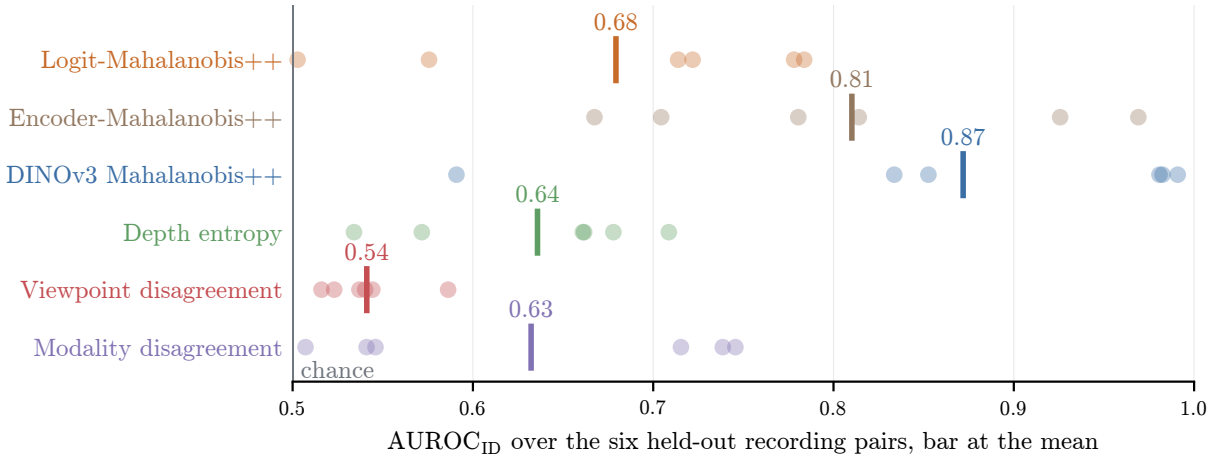


Figure 27: $\mathrm{AUROC}_{\mathrm{ID}}$ of every detector: one dot per pair of held-out in-distribution recordings, bar at the mean over the six pairs. A detector that responds to distribution shift alone would sit at chance (0.5).

As visible in Figure 27, the two image-feature-based detectors (Encoder-Mahalanobis++, DINOv3 Mahalanobis++) do rate the held-out ID recordings as OOD, while the other detectors rate them closer to ID. This suggests that the image-based detectors separate individual recordings or scenes, not ID and OOD.

In the following, we report $\mathrm{AUROC}_{\mathrm{ID}}$ alongside the results, either directly or as the baseline-corrected AUROC, which is, for a given tier, $\mathrm{AUROC}_{\text{OOD-Tier}} - \mathrm{AUROC}_{\mathrm{ID}}$. A positive value indicates that the detector separates the tier from held-out ID recordings, while a negative value indicates that it separates recordings, not ID and OOD.

10.4.2 OOD Detection Results

Figure 28 and Figure 29 show the performance of every detector w.r.t. the metrics introduced in Section 10.2. Figure 30 shows the baseline-corrected AUROC for direct comparison.

The result metrics in Figure 29 and Figure 28 suggest that the visual detectors (Encoder-Mahalanobis++, DINOv3 Mahalanobis++) perform best on all tiers by a considerable margin. However, as discussed in Section 10.4.1, these detectors separate individual recordings, not just ID and OOD. When comparing this to the baseline-corrected AUROC in Figure 30, the ordering changes slightly. We investigate this in full detail in Section 10.5.

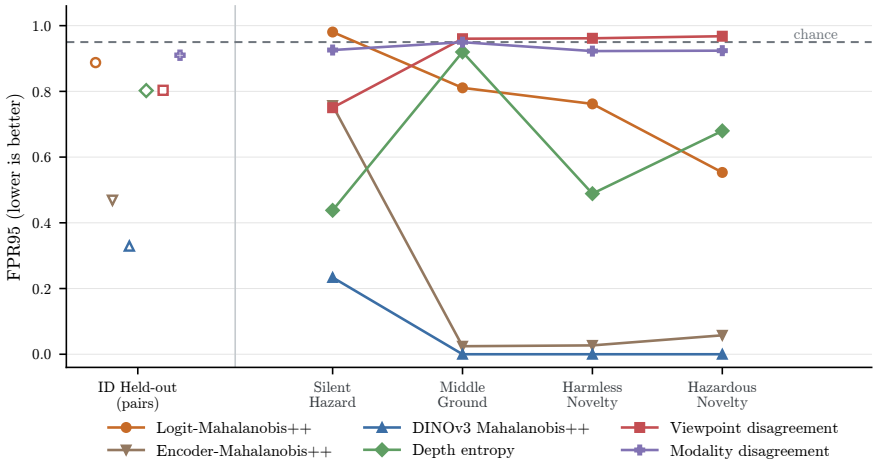


Figure 28: FPR95 of every detector against the held-out in-distribution reference, with the tiers pooled into the quadrants of Section 10.1. Color and marker identify the detector, and the dashed line is where a chance detector sits. The left column repeats the held-out ID score of Section 10.4.1.

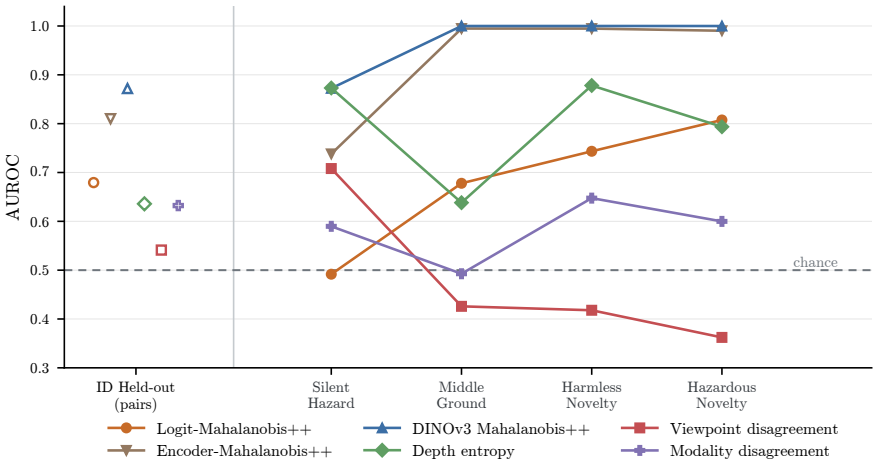


Figure 29: Detection AUROC of every detector against the held-out in-distribution reference, drawn as in Figure 28.

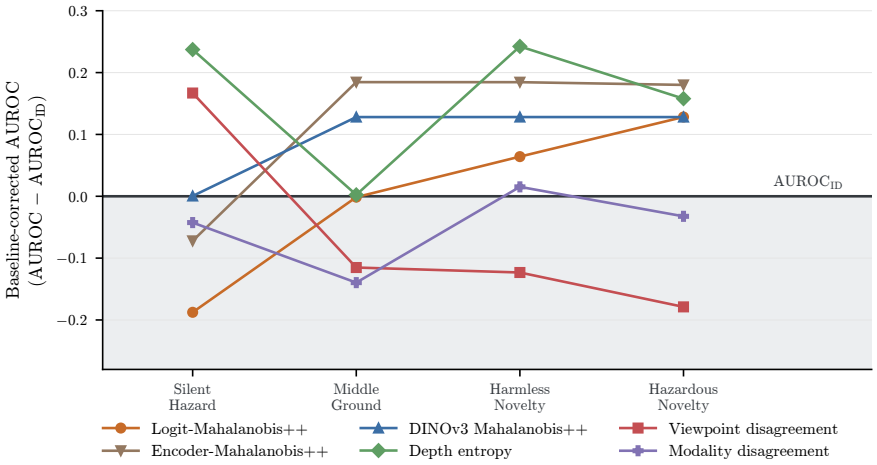


Figure 30: The same AUROC minus each detector's own $\mathrm{AUROC}_{\mathrm{ID}}$, drawn as in Figure 28. In the shaded region below the line the detector separates recordings rather than distributions.

10.5 Failure Mode Analysis and Discussion

To investigate the failure modes, we first visualize the baseline-corrected AUROC (Section 10.4.1) in the four quadrants defined in Figure 18. The following subsections refer to this figure to discuss strengths and failure modes of the individual detectors.

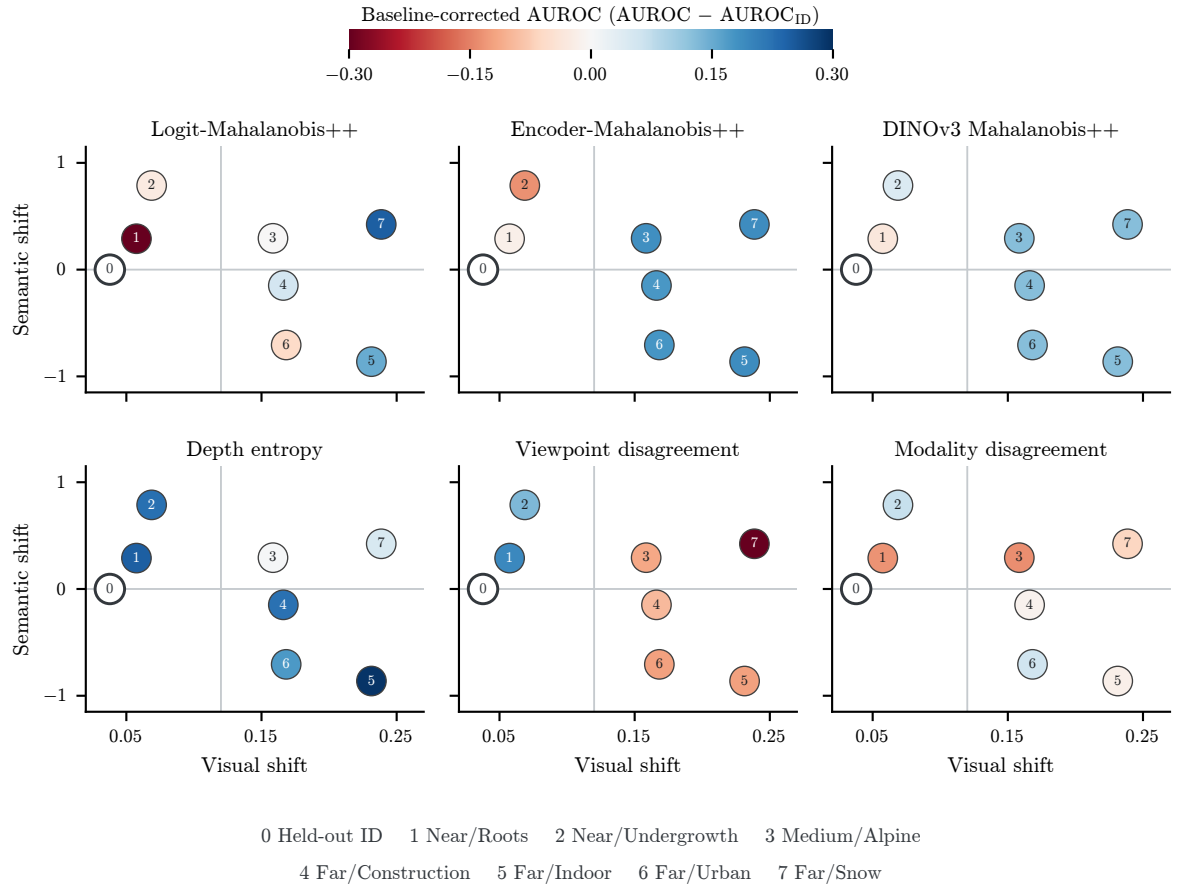


Figure 31: Baseline-corrected AUROC per detector, one panel each, with every tier at its measured position from Figure 18. All panels show the same seven tiers, numbered as in the key below, and color is that tier's AUROC over the panel's own AUROC$_{ID}$. Tier 0 is the held-out ID reference AUROC$_{ID}$ is measured on, so it carries no color.

10.5.1 Logit-Mahalanobis++

Logit-Mahalanobis++ only flags the most visually novel tiers as OOD and completely misses the others. We investigate the feature space by visualizing it via PCA in Figure 32. At least in the first three principal components, no tier occupies a region of the BEV feature space that the held-out in-distribution cells do not already cover, and all of them concentrate in the dense part of the ID cloud. This is consistent with the low baseline-corrected AUROC of Logit-Mahalanobis++, indicating that the BEV feature space is not well suited for OOD detection.



Figure 32: The ℓ_2-normalized BEV cell features Logit-Mahalanobis++ reads, reduced to their first three principal components. Each panel overlays the cells of one OOD tier on the held-out ID cells. The three components carry 44% of the in-distribution variance, so a tier can sit off the space they span.

10.5.2 Methods Based on Visual Features (Encoder-Mahalanobis++, DINOv3 Mahalanobis++)

As expected, the two visual-feature-based methods (Encoder-Mahalanobis++, DINOv3 Mahalanobis++) perform well along the visual axis. The visual shift definition involves DINOv3 features, so it would be very surprising if the DINOv3 method performed poorly on this axis. However, both fail to separate the silent hazard tier from ID recordings, which suggests that more subtle visual cues for traversability are not taken into account. This does not completely invalidate them as OOD detection methods, but is a major limitation for safety-critical applications, where silent hazards are important to detect.

10.5.3 Depth Entropy

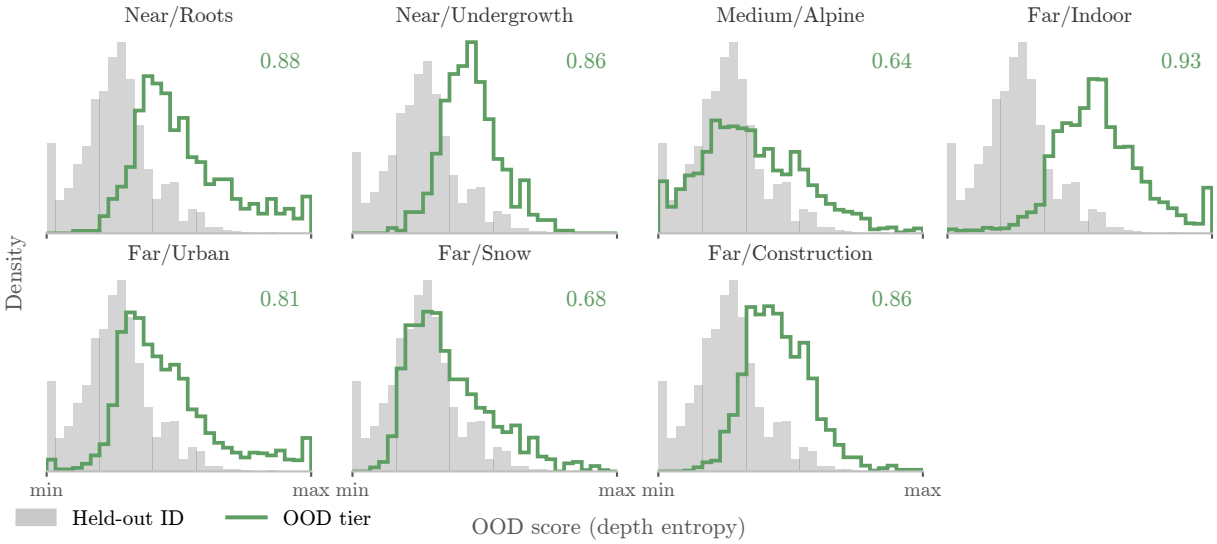


Figure 33: Depth entropy per tier against the held-out in-distribution reference (gray). The number in a panel is that tier's detection AUROC, grayed out where it does not clear the detector's $\mathrm{AUROC}_{\mathrm{ID}}$. Depth entropy has no absolute scale, so the axis runs from the 1st to the 99th percentile over all tiers.

The depth entropy method performs well on terrain that is geometrically complex in novel ways, but misses the Snow and Urban tiers, which are visually novel but rather simple in geometry. Compared to its $\mathrm{AUROC}_{\mathrm{ID}}$, this detector performs best overall, but is not usable on its own, because it essentially misses the *visually novel and hazardous* tier.

10.5.4 Viewpoint Disagreement

The results of viewpoint disagreement are interesting, because the method partially detects the shift in traversability on visually familiar terrain, but completely fails to detect visually novel terrain as OOD (Figure 34).

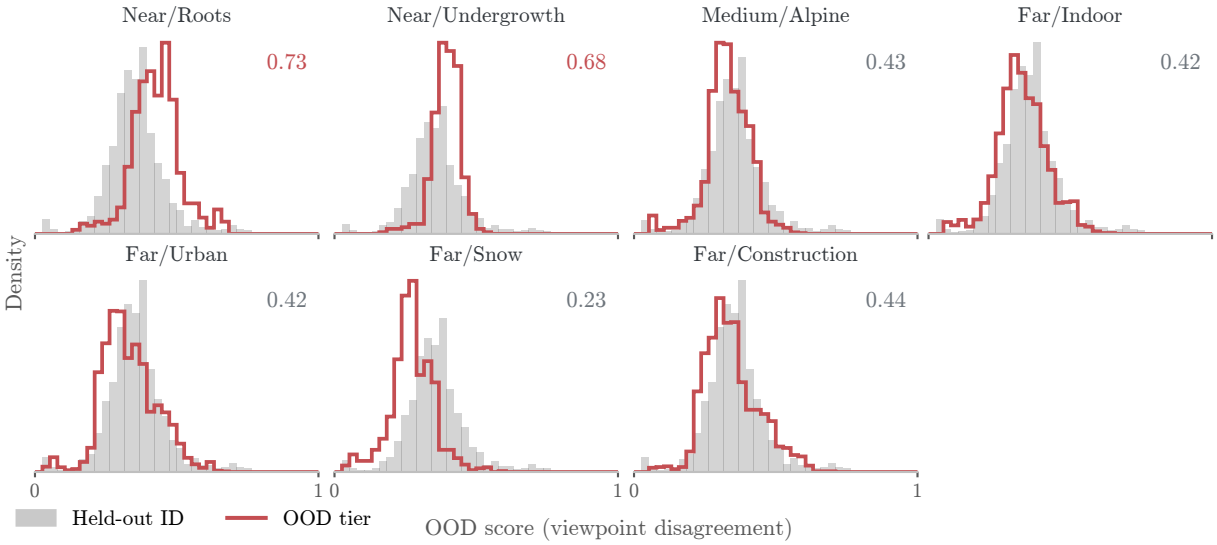


Figure 34: Viewpoint disagreement per tier, drawn as in Figure 33. The score is a spread in traversability units, so the axis is the full output range.

The histograms in Figure 34 show that prediction disagreement is often lower for visually novel terrain, suggesting that the predicted traversability has a lower variance there. Figure 35 confirms this: The predicted traversability distribution is more concentrated for visually novel terrain, which leads us to the assumption that the model collapses to a narrower range of predictions in unfamiliar situations.

In theory, the distribution of predictions could be used as an OOD signal, since every tier is narrower than the held-out ID reference. In practice, it is not usable as one: The spread is a property of a set of windows rather than of a single window, and the mean moves in both directions, so an individual prediction cannot be placed against the reference.

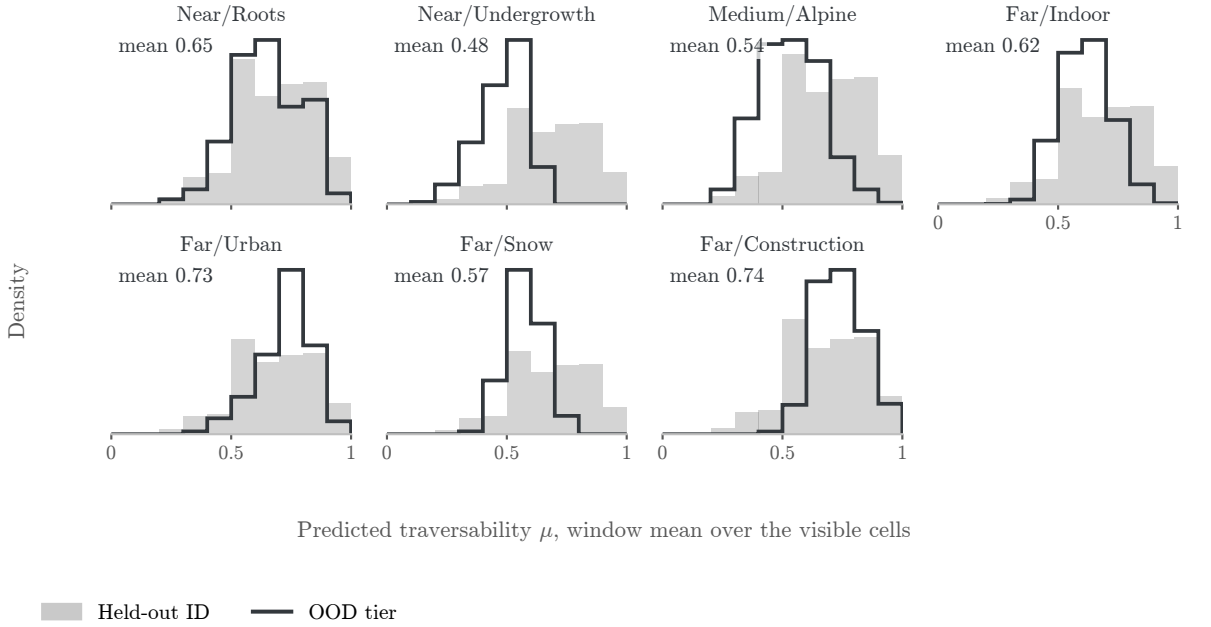


Figure 35: Predicted traversability per tier against the held-out ID reference (gray), in ten buckets over the output range. The number in a panel is that tier's mean.

10.5.5 Modality Disagreement

Modality disagreement is a clear negative result: Figure 36 shows that the disagreement between RGB and RGB+D predictions is very small across all tiers, suggesting that our trained model relies purely on the RGB branch and does not use the depth information.

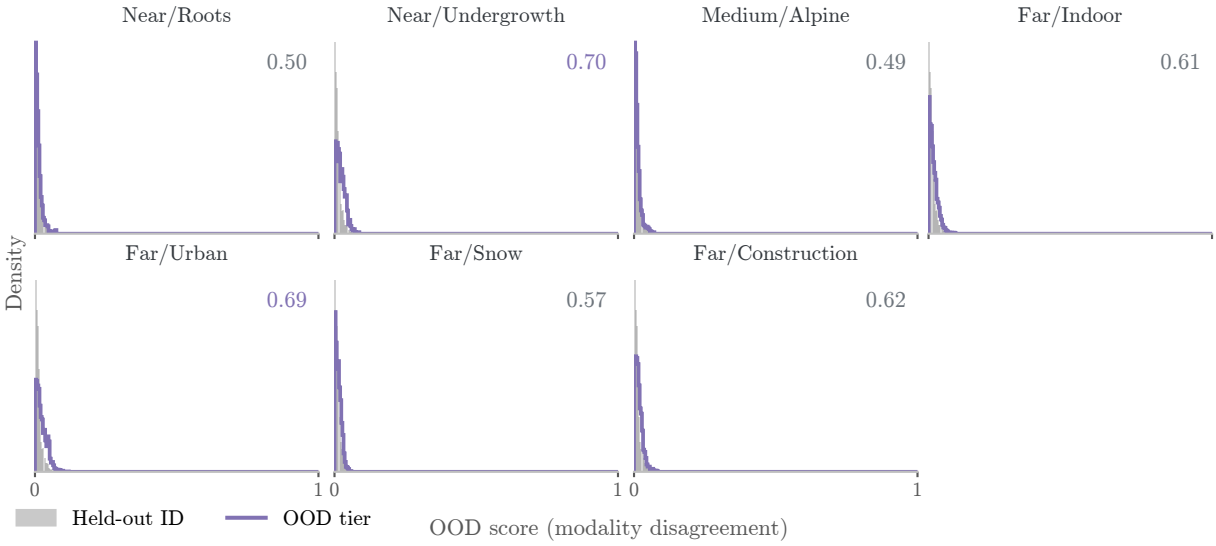


Figure 36: Modality disagreement per tier, drawn as in Figure 34.

10.6 Ensembling to Mitigate Failure Modes

It is important to mention that this ensemble approach was thought of *after* the individual methods were evaluated. We do not directly tune the ensemble against the evaluation method (an example of this would be choosing different weights for the components), but the choice of components itself *is* influenced by the evaluation method. As a result, performance on the evaluated tiers is not entirely independent.

For this reason, we describe and report the ensemble method here, separate from the individual methods.

10.6.1 Ensemble-Method

The failure modes of the single methods are complementary. Based on this finding, we combine three detectors of different kinds: Encoder-Mahalanobis++ as an image-feature-based detector, depth entropy as a geometry-based detector, and viewpoint disagreement as a semantic-based detector.

The three scores are on different scales, so we cannot average them directly. For each method m, we take its scores over the in-distribution training windows and compute their mean $\bar{s}_m$ and standard deviation σ_m. Every score $s_m(w)$ of a window w is then expressed in standard deviations from that in-distribution mean, and the ensemble score is the average of the three:

$$s_{\text{ens}}(w) = \frac{1}{3} \sum_{m} \frac{s_m(w) - \bar{s}_m}{\sigma_m}$$

Both statistics are read off training windows only, so no OOD recording enters the fitting process. Using equal weights avoids tuning against the tiers we evaluate on. A window is flagged when s_{ens} exceeds a threshold, which we set as a percentile of the training-window scores. No method in the ensemble requires additional forward passes through the model, so the method is computationally cheap.

10.6.2 Results of the Ensemble



Figure 37: The ensemble on the measured shift plane, drawn as one panel of Figure 31: position is the tier, color is the AUROC over the ensemble's own AUROC$_{ID}$.

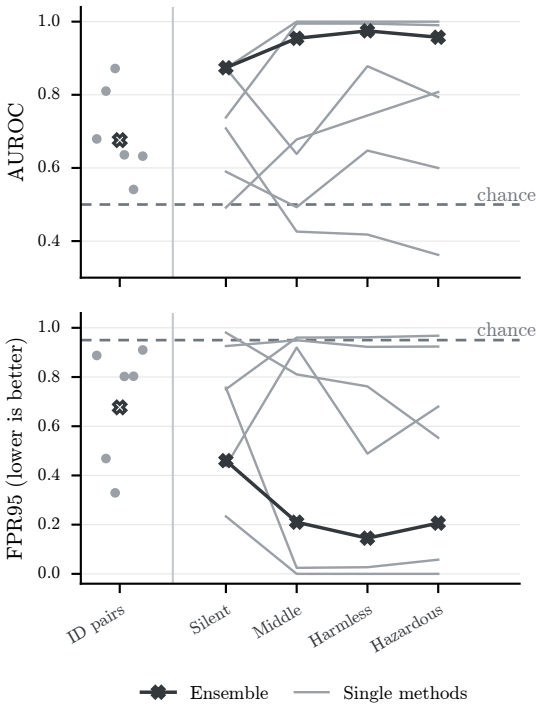


Figure 38: AUROC and FPR95 of the ensemble per quadrant, drawn as in Figure 28, with the six single methods in gray behind it.

The ensemble is the only method that clears its own AUROC$_{ID}$ of 0.68 in all four quadrants (Figure 37): +0.20 on the silent hazards and +0.28 to +0.30 where the terrain also looks new, against +0.00 in the weakest quadrant of depth entropy, the best single method. Figure 38 shows the two untransformed metrics behind this.

Whether this is usable depends on what the alarm is for. Setting the threshold at a 5% false-alarm budget flags 4% of the held-out ID windows and 74% to 87% of the windows in the three shifted-appearance quadrants, which is enough to warn that the estimator has left its training domain. It is not enough for a hazard warning: only 41% of the silent hazard windows are flagged.

10.7 Conclusion from the Failure Mode Analysis

No single detector clears its AUROC$_{ID}$ in all four quadrants, and the failure modes are complementary: methods detecting visual novelty miss the silent hazards, and vice versa.

Methods that rely on features from earlier stages of the architecture (Encoder-Mahalanobis++, DINOv3 Mahalanobis++, Depth Entropy) generally perform better than methods that rely on features from later stages (Logit-Mahalanobis++, Disagreement). This suggests that the feature space at later stages is poorly suited for OOD detection.

10.8 Answers to RQ2-RQ6

In the following, we summarize the results to answer RQ2-RQ6.

RQ2 Logit-Mahalanobis++ flags only the visually novel quadrants (0.81 against $\mathrm{AUROC}_{\mathrm{ID}} = 0.68$) and is at chance on silent hazards. Encoder-Mahalanobis++, the same distance on the encoder features, reaches 0.99.

RQ3 Mahalanobis++ on DINOv3 features saturates on visual novelty (1.00) but carries the highest $\mathrm{AUROC}_{\mathrm{ID}}$ (0.87) and misses silent hazards.

RQ4 Depth entropy is the only detector that scores higher on the silent hazard quadrant than on held-out ID recordings (0.87 against $\mathrm{AUROC}_{\mathrm{ID}} = 0.64$), and the weakest detector on terrain that is visually novel but geometrically simple.

RQ5 Viewpoint disagreement detects the silent hazards (0.71 against $\mathrm{AUROC}_{\mathrm{ID}} = 0.54$) and drops below chance on visually novel terrain.

RQ6 Modality disagreement is a negative result: The prediction barely changes without the depth branch, likely because the model does not use depth information.

Combining the three methods with complementary failure modes (Section 10.6) covers all four quadrants and performs worst on the silent hazards quadrant.

11 Conclusion and Future Work

11.1 Conclusion

Trained on the forest recordings of GrandTour under proprioceptive labels, WayFASTER does not generalize across recordings (RQ1): The predictions contract towards the label mean within the first epoch, and no regularization class we tested moves the calibration slope beyond the seed noise of the training runs (Section 8.2.4). The supervision seems to be the primary constraint, since the same pipeline reproduces the published behavior on the WayFASTER dataset and collapses on that same data once its labels are resampled to the GrandTour distribution (Section 8.4). GrandTour carries a strong bias towards traversable terrain, which no proprioceptive scheme can correct, because the robot never walks on the terrain that is missing. As a side result, we find that training, validation, and field trials of the published WayFASTER results partly cover the same ground (Section 8.5), so none of them isolates generalization to an unseen environment.

A small set of relative human comparisons, which does not carry this bias, is enough to train an estimator that outperforms the proprioceptively supervised one on the human labels, at a fraction of W-RIZZ's label budget (Table 8).

On that estimator, no post-hoc detector separates every group of shifted recordings from held-out in-distribution ones (RQ2-RQ6). The appearance-based detectors are reliable on novel-looking terrain and at chance on forest that merely became harder to traverse, and depth entropy and viewpoint disagreement fail the other way around. Modality disagreement is a negative result, since the estimator barely uses its depth branch. An equally weighted ensemble of complementary detectors is the only method that clears its own in-distribution baseline in every quadrant of the shift plane (Section 10.6).

11.2 Takeaways

We list takeaways that we consider to generalize beyond the setup of this work.

Architecture choice Selecting a traversability estimation architecture is difficult since the field has neither a shared benchmark nor a shared label definition. Its suitability can therefore only be assessed after training and evaluating it on the dataset at hand.

GrandTour Despite its excellent sensor coverage and diversity, the GrandTour dataset is unsuited for generating proprioceptive traversability labels without using a strong inductive bias. This is due to its selection bias towards traversable terrain.

Label imbalance A label distribution as skewed as GrandTour's cannot be compensated by the standard remedies for imbalanced datasets, such as inverse-frequency weighting or resampling (Chapter 8), since neither of them introduces observations of the underrepresented terrain.

Feature choice for OOD detection In our case, features from earlier stages of the network separate novel terrain more reliably than features from later stages (Section 10.4).

Complementary failure modes The OOD detectors fail on different kinds of distribution shift. An ensemble of OOD detectors covering different modalities can mitigate their individual weaknesses (Section 10.6).

11.3 Limitations of the Work

Scale of the dataset The ID set is limited to the 1 h 40 min of forest recordings in the GrandTour dataset, on one platform and during one season.

No test set As mentioned in Section 4.2, checkpoint selection and final evaluation share the validation split. This was an artifact of the limited ID data available. It is common in the OOD literature [1], [45] to not have a test set. Usually, the detector is evaluated in a closed-loop task, and only a success rate is reported.

No closed-loop evaluation Evaluating the trained OOD detectors on a robot is out of scope for this work. This means that we cannot measure how well they improve the robot's performance.

Single annotator for human labels The human labels in Section 9.3 come from one annotator only, which means there might be a bias in the labels that is not captured by the self-consistency checks.

Estimator bias As a consequence of the above point, the evaluation of the OOD detectors is also biased by the same annotator's decision preferences.

11.4 Future Work

This section describes the directions we consider most promising for future work.

11.4.1 Establish a Traversability Estimation Benchmark

The field of traversability estimation lacks a shared benchmark, which is in part due to the lack of a shared label definition. This makes it hard to compare different approaches and to select a state of the art, and the same holds for the OOD detectors built on top of them. Closed-loop evaluation on a robot is the usual substitute, but a reported success rate is too coarse to rank detectors against each other. Establishing such a benchmark would involve curating a diverse set of datasets, agreeing on one or more label definitions, and collecting human labels as a reference that does not depend on the platform.

11.4.2 Use Novel Datasets with More Low-Traversability Examples

The recent appearance of lower-cost quadruped robots, like the Unitree Go2, makes it more feasible to collect low-traversability data, which is a scarce resource in existing quadruped datasets. One example is the ForEnt dataset [41], as mentioned in Section 3.4, which contains a large number of low-traversability examples, but was not yet fully available at the time of writing. Using datasets like ForEnt could help improve the performance of traversability estimators trained on proprioceptive labels.

11.4.3 Compare Different OOD Methods for Traversability Estimators That Rely on the *Unknown* → *Untraversable* Assumption

As mentioned in Chapter 9, one way of handling the problem of label sparsity and selection bias is by giving the model a strong inductive bias that terrain the robot has never encountered is untraversable, as done by Wild Visual Navigation (WVN) [13], among others. They use an OOD detector (the reconstruction loss of an autoencoder) to pull the label to 0 (untraversable) for patches that have never been observed. It would be interesting to compare and evaluate different OOD methods against a method that relies on this assumption.

11.4.4 Augment Proprioceptive Labels With Relative Human Labels

The two labeling schemes used in this work, the proprioceptive labels and the relative human labels, have complementary strengths and weaknesses. Proprioceptive labels are abundant and have an absolute scale, but they carry a strong selection bias for traversable terrain. Relative human labels are comparatively scarce and carry only a relative scale, but they are not selection-biased. Combining a smaller set of relative labels with proprioceptive labels to exploit their respective strengths and mitigate their weaknesses could be beneficial.

In our experiments, we trained several models with a weighted loss that combined both label types, but the results were inconclusive against the HDR metric and the metrics for the RHE labels (in that they did not surpass the best-performing model on the respective metric).

Further research could try to find a better way to combine the two label types. This would also involve designing an appropriate way to evaluate these hybrid models.

Bibliography

[1] M. V. Gasparino, A. N. Sivakumar, and G. Chowdhary, "WayFASTER: A Self-Supervised Traversability Prediction for Increased Navigation Awareness," Mar. 2024, doi: 10.48550/arXiv.2402.00683.

[2] M. G. Castro *et al.*, "How Does It Feel? Self-Supervised Costmap Learning for Off-Road Vehicle Traversability," Feb. 2023, doi: 10.48550/arXiv.2209.10788.

[3] M. Patel *et al.*, "TartanGround: A Large-Scale Dataset for Ground Robot Perception and Navigation," Jul. 2025, doi: 10.48550/arXiv.2505.10696.

[4] J. Frey *et al.*, "GrandTour: A Legged Robotics Dataset in the Wild for Multi-Modal Perception and State Estimation," Feb. 2026, doi: 10.48550/arXiv.2602.18164.

[5] Oxford English Dictionary, "Aleatoric, Adj.." Oxford University Press, Jul. 2023. [Online]. Available: https://doi.org/10.1093/OED/9699485804

[6] Oxford English Dictionary, "Epistemic, Adj.." Oxford University Press, Sep. 2025. [Online]. Available: https://doi.org/10.1093/OED/4330537275

[7] J. Yang *et al.*, "OpenOOD: Benchmarking Generalized Out-of-Distribution Detection," Oct. 2022, doi: 10.48550/arXiv.2210.07242.

[8] A. Miyai *et al.*, "Generalized Out-of-Distribution Detection and Beyond in Vision Language Model Era: A Survey," Jun. 2025, doi: 10.48550/arXiv.2407.21794.

[9] C. Li, E. Zhang, C. Geng, and S. Chen, "Recent Advances in Out-of-Distribution Detection with CLIP-Like Models: A Survey," May 2025, doi: 10.48550/arXiv.2505.02448.

[10] S. Lu, Y. Wang, L. Sheng, L. He, A. Zheng, and J. Liang, "Out-of-Distribution Detection: A Task-Oriented Survey of Recent Advances," Aug. 2025, doi: 10.48550/arXiv.2409.11884.

[11] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, Jun. 2006, doi: 10.1016/j.patrec.2005.10.010.

[12] Q. Zhang, Q. Feng, J. T. Zhou, Y. Bian, Q. Hu, and C. Zhang, "The Best of Both Worlds: On the Dilemma of Out-of-distribution Detection," Oct. 2024, doi: 10.48550/arXiv.2410.11576.

[13] J. Frey, M. Mattamala, N. Chebrolu, C. Cadena, M. Fallon, and M. Hutter, "Fast Traversability Estimation for Wild Visual Navigation," May 2023, doi: 10.48550/arXiv.2305.08510.

[14] M. Caron *et al.*, "Emerging Properties in Self-Supervised Vision Transformers," May 2021, doi: 10.48550/arXiv.2104.14294.

[15] Y. Jeon, E. I. Son, and S.-W. Seo, "Follow the Footprints: Self-supervised Traversability Estimation for Off-road Vehicle Navigation Based on Geometric and Visual Cues," Feb. 2024, doi: 10.48550/arXiv.2402.15363.

[16] A. Kirillov *et al.*, "Segment Anything," Apr. 2023, doi: 10.48550/arXiv.2304.02643.

[17] S. Jung, J. Lee, X. Meng, B. Boots, and A. Lambert, "V-STRONG: Visual Self-Supervised Traversability Learning for Off-road Navigation," Mar. 2024, doi: 10.48550/arXiv.2312.16016.

[18] S. Ægidius, D. Hadjivelichkov, J. Jiao, J. Embley-Riches, and D. Kanoulas, "Watch Your STEPP: Semantic Traversability Estimation Using Pose Projected Features," Jan. 2025, doi: 10.48550/arXiv.2501.17594.

[19] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," Feb. 2024, doi: 10.48550/arXiv.2304.07193.

[20] S. H. Khorasgani, Y. Chen, and F. Shkurti, "SLIC: Self-Supervised Learning with Iterative Clustering for Human Action Videos," Jun. 2022, doi: 10.48550/arXiv.2206.12534.

[21] J. Gao, M. Chen, L. Xiang, and C. Xu, "A Comprehensive Survey on Evidential Deep Learning and Its Applications," Sep. 2024, doi: 10.48550/arXiv.2409.04720.

[22] S. Gupta *et al.*, "Out-of-Distribution Detection for Continual Learning: Design Principles and Benchmarking," Dec. 2025, doi: 10.48550/arXiv.2512.19725.

[23] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, Jul. 2009, doi: 10.1145/1541880.1541882.

[24] X. Tao, X. Gong, X. Zhang, S. Yan, and C. Adak, "Deep Learning for Unsupervised Anomaly Localization in Industrial Images: A Survey," *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–21, 2022, doi: 10.1109/TIM.2022.3196436.

[25] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," Oct. 2018, doi: 10.48550/arXiv.1610.02136.

[26] K. Lee, K. Lee, H. Lee, and J. Shin, "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks," Oct. 2018, doi: 10.48550/arXiv.1807.03888.

[27] J. Ren, S. Fort, J. Liu, A. G. Roy, S. Padhy, and B. Lakshminarayanan, "A Simple Fix to Mahalanobis Distance for Improving Near-OOD Detection," Jun. 2021, doi: 10.48550/arXiv.2106.09022.

[28] E. Nalisnick, A. Matsukawa, Y. W. Teh, D. Gorur, and B. Lakshminarayanan, "Do Deep Generative Models Know What They Don't Know?," Feb. 2019, doi: 10.48550/arXiv.1810.09136.

[29] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning," Oct. 2016, doi: 10.48550/arXiv.1506.02142.

[30] A. Vyas, N. Jammalamadaka, X. Zhu, D. Das, B. Kaul, and T. L. Willke, "Out-of-Distribution Detection Using an Ensemble of Self Supervised Leave-out Classifiers," Sep. 2018, doi: 10.48550/arXiv.1809.03576.

[31] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles," Nov. 2017, doi: 10.48550/arXiv.1612.01474.

[32] Y. Fathullah and M. J. F. Gales, "Self-Distribution Distillation: Efficient Uncertainty Estimation," Mar. 2022, doi: 10.48550/arXiv.2203.08295.

[33] M. Sensoy, L. Kaplan, and M. Kandemir, "Evidential Deep Learning to Quantify Classification Uncertainty," Oct. 2018, doi: 10.48550/arXiv.1806.01768.

[34] A. Amini, W. Schwarting, A. Soleimany, and D. Rus, "Deep Evidential Regression," Nov. 2020, doi: 10.48550/arXiv.1910.02600.

[35] Y. Ming, Z. Cai, J. Gu, Y. Sun, W. Li, and Y. Li, "Delving into Out-of-Distribution Detection with Vision-Language Representations," *Advances in Neural Information Processing Systems*, vol. 35, pp. 35087–35102, Dec. 2022, Accessed: Mar. 02, 2026. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/e43a33994a28f746dcfd53eb51ed3c2d-Abstract-Conference.html

[36] P. Sinhamahapatra, F. Schwaiger, S. Bose, H. Wang, K. Roscher, and S. Guennemann, "Finding Dino: A Plug-and-Play Framework for Zero-Shot Detection of Out-of-Distribution Objects Using Prototypes," Feb. 2025, doi: 10.48550/arXiv.2404.07664.

[37] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," Feb. 2021, doi: 10.48550/arXiv.2103.00020.

[38] W. Chen, O. Mees, A. Kumar, and S. Levine, "Vision-Language Models Provide Promptable Representations for Reinforcement Learning," May 2024, doi: 10.48550/arXiv.2402.02651.

[39] X. Cai *et al.*, "EVORA: Deep Evidential Traversability Learning for Risk-Aware Off-Road Autonomy," Mar. 2024, doi: 10.48550/arXiv.2311.06234.

[40] M. Hutter *et al.*, "ANYmal - a Highly Mobile and Dynamic Quadrupedal Robot," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2016, pp. 38–44. doi: 10.1109/IROS.2016.7758092.

[41] N. Kirdwichai and D. Tarapore, "ForEnt: A Multi-Modal Dataset for Characterizing Quadruped Robot Entrapments in Forest Environments," Jun. 2026, doi: 10.48550/arXiv.2606.19675.

[42] J. Frey *et al.*, "Boxi: Design Decisions in the Context of Algorithmic Performance for Robotics," Apr. 2025, doi: 10.48550/arXiv.2504.18500.

[43] K. Chen, R. Nemiroff, and B. T. Lopez, "Direct LiDAR-Inertial Odometry: Lightweight LIO with Continuous-Time Motion Correction," Mar. 2023, doi: 10.48550/arXiv.2203.03749.

[44] B. Wen, M. Trepte, J. Aribido, J. Kautz, O. Gallo, and S. Birchfield, "FoundationStereo: Zero-Shot Stereo Matching," Apr. 2025, doi: 10.48550/arXiv.2501.09898.

[45] M. V. Gasparino *et al.*, "WayFAST: Navigation with Predictive Traversability in the Field," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10651–10658, Oct. 2022, doi: 10.1109/LRA.2022.3193464.

[46] E. Steyerberg *et al.*, "Assessing the Performance of Prediction Models A Framework for Traditional and Novel Measures," *Epidemiology (Cambridge, Mass.)*, vol. 21, pp. 128–138, Jan. 2010, doi: 10.1097/EDE.0b013e3181c30fb2.

[47] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," Aug. 2017, doi: 10.48550/arXiv.1706.04599.

[48] J. A. Hanley and B. J. McNeil, "The Meaning and Use of the Area under a Receiver Operating Characteristic (ROC) Curve.," *Radiology*, vol. 143, no. 1, pp. 29–36, Apr. 1982, doi: 10.1148/radiology.143.1.7063747.

[49] D. R. Cox, "Two Further Applications of a Model for Binary Regression," *Biometrika*, vol. 45, no. 3–4, pp. 562–565, 1958, doi: 10.1093/biomet/45.3-4.562.

[50] Y. Yang, K. Zha, Y.-C. Chen, H. Wang, and D. Katabi, "Delving into Deep Imbalanced Regression," 2021, doi: 10.48550/ARXIV.2102.09554.

[51] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, "The Balanced Accuracy and Its Posterior Distribution," in *2010 20th International Conference on Pattern Recognition*, Istanbul, Turkey: IEEE, Aug. 2010, pp. 3121–3124. doi: 10.1109/ICPR.2010.764.

[52] J. Ren, M. Zhang, C. Yu, and Z. Liu, "Balanced MSE for Imbalanced Visual Regression," Mar. 2022, doi: 10.48550/arXiv.2203.16427.

[53] N. Keetha *et al.*, "AnyLoc: Towards Universal Visual Place Recognition," Nov. 2023, doi: 10.48550/arXiv.2308.00688.

[54] S. Kiritchenko and S. Mohammad, "Best-Worst Scaling More Reliable than Rating Scales: A Case Study on Sentiment Intensity Annotation," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, R. Barzilay and M.-Y. Kan, Eds., Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 465–470. doi: 10.18653/v1/P17-2074.

[55] R. D. Luce, R. M. Nosofsky, D. M. Green, and A. F. Smith, "The Bow and Sequential Effects in Absolute Identification," *Perception & Psychophysics*, vol. 32, no. 5, pp. 397–408, Sep. 1982, doi: 10.3758/BF03202769.

[56] A. Schreiber, A. N. Sivakumar, P. Du, M. V. Gasparino, G. Chowdhary, and K. Driggs-Campbell, "W-RIZZ: A Weakly-Supervised Framework for Relative Traversability Estimation in Mobile Robotics," Jun. 2024, doi: 10.48550/arXiv.2406.02822.

[57] D. Zoran, P. Isola, D. Krishnan, and W. T. Freeman, "Learning Ordinal Relationships for Mid-Level Vision," in *2015 IEEE International Conference on Computer Vision (ICCV)*, Dec. 2015, pp. 388–396. doi: 10.1109/ICCV.2015.52.

[58] A. Tarvainen and H. Valpola, "Mean Teachers Are Better Role Models: Weight-averaged Consistency Targets Improve Semi-Supervised Deep Learning Results," Apr. 2018, doi: 10.48550/arXiv.1703.01780.

[59] G. Humblot-Renaux, S. Escalera, and T. B. Moeslund, "Beyond AUROC & Co. for Evaluating out-of-Distribution Detection Performance," Jun. 2023, doi: 10.48550/arXiv.2306.14658.

[60] M. Mueller and M. Hein, "Mahalanobis++: Improving OOD Detection via Feature Normalization," May 2025, doi: 10.48550/arXiv.2505.18032.

[61] O. Siméoni *et al.*, "DINOv3," Aug. 2025, doi: 10.48550/arXiv.2508.10104.

Declaration on the Use of Generative AI

In the preparation of this Master's thesis, generative artificial intelligence tools (Claude Sonnet 5, Claude Opus 5, GitHub Copilot) were used for the following tasks:

- Explore content such as research papers and code, and brainstorm ideas.
- Write, prototype, and debug code snippets, primarily for exploratory prototypes, plotting and visualization code, and refactoring.
- Improve language quality, style, and clarity. In all instances, a draft version was provided by the author and was then refined or written out into prose with AI assistance.

All outputs generated by AI were critically reviewed, verified, and adapted by the author. The author takes full responsibility for the entire content, the findings, and the conclusions presented in this thesis.

Appendix A
RHE Hyperparameter Tuning

		Range	Start	Optimum	Adopted
$P_x[0,1]$	position continuity	$[0.001, 5]$	1	1.657	1.5
$P_x[2]$	heading continuity	$[0.001, 5]$	1	0.931	1
$P_m[\mu]$	label smoothness	$[0.001, 5]$	0.25	0.205	0.2
$P_m[\nu]$		$[0.001, 5]$	0.25	0.156	0.2
$P_m[\delta\theta]$	heading-bias smoothness	$[0.001, 5]$	1.5	1.636	1.5
$P_w[0,1]$	position residual	$[0.001, 5]$	1	1.212	1.2
$P_w[2]$	heading residual	$[0.001, 5]$	1	1.009	1
λ_{tie}	tying strength	$[0.01, 5]$	1.3	1.290	1.3
Held fixed: $N = 10$, step 2, $\Delta t = 0.1$ s, $x_{\text{lo}} = 0.01$, $x_{\text{hi}} = 0.1$					

Table 10: The eight tuned RHE cost weights of Section 6.1: search range, starting point, differential-evolution optimum, and the rounded values the pipeline runs with. $P_x[0,1]$ and $P_w[0,1]$ are one tied variable each, by the planar x/y symmetry.

Scenario	MAE μ	MAE ν	$\overline{o}_\mu$	$\overline{o}_\nu$
Excitation ($\sigma = 0.02$ m)				
constant terrain	0.014	0.024	1.00	1.00
terrain transition	0.015	0.025	1.00	1.00
mixed excitation	0.023	0.028	0.76	0.76
mission profile	0.017	0.038	0.87	0.28
step ramp	0.065	0.054	0.81	0.81
straight walking ($\omega = 0$)	0.014	0.049	1.00	0.00
turning in place ($v = 0$)	0.055	0.017	0.00	1.00
full stop ($v = \omega = 0$)	0.300	0.250	0.00	0.00
Odometry noise (constant terrain)				
$\sigma = 0.01$ m	0.007	0.012	1.00	1.00
$\sigma = 0.02$ m	0.014	0.024	1.00	1.00
$\sigma = 0.05$ m	0.035	0.057	1.00	1.00
$\sigma = 0.1$ m	0.070	0.104	1.00	1.00
$\sigma = 0.2$ m	0.127	0.186	1.00	1.00

Table 11: Mean absolute error of the RHE against the known parameters of each synthetic scenario, with the mean observability scores $\overline{o}_\mu$, $\overline{o}_\nu$. σ is the standard deviation of the Gaussian noise added to the simulated odometry.

Appendix B
GrandTour Tint Correction



Figure 39: Red-tinted frames (left) and the corrected output (right), for two affected missions.